

Cluster_based and Explainable ML Model Framework (Co2 emission)

```
In [2]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

import plotly.express as px
import geopandas as gpd
from matplotlib.lines import Line2D

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering
from sklearn.cluster import DBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.cluster import SpectralClustering

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from interpret.glassbox import ExplainableBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error

from interpret.glassbox import ExplainableBoostingRegressor
from interpret import show
import shap
from sklearn.inspection import PartialDependenceDisplay
```

C:\Users\USER\AppData\Local\Programs\Orange\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

1.Exploratory Data Analysis

1.1 Data Collection and Understanding

```
In [5]: df = pd.read_csv("owid-co2-data.csv")
df
```

Out[5]:

	country	year	iso_code	population	gdp	cement_co2	cement_co2_per_capita	co2	co2_growth_abs	co2_growth_prct	...	share_global_other_co2	share_of_temperature_change_from_ghg	temper
0	Afghanistan	1750	AFG	2802560.0	NaN	0.000	0.000	NaN	NaN	NaN	...	NaN	NaN	
1	Afghanistan	1751	AFG	NaN	NaN	0.000	NaN	NaN	NaN	NaN	...	NaN	NaN	
2	Afghanistan	1752	AFG	NaN	NaN	0.000	NaN	NaN	NaN	NaN	...	NaN	NaN	
3	Afghanistan	1753	AFG	NaN	NaN	0.000	NaN	NaN	NaN	NaN	...	NaN	NaN	
4	Afghanistan	1754	AFG	NaN	NaN	0.000	NaN	NaN	NaN	NaN	...	NaN	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
50186	Zimbabwe	2019	ZWE	15271377.0	2.514642e+10	0.473	0.031	10.263	-0.942	-8.411	...	NaN	0.106	
50187	Zimbabwe	2020	ZWE	15526888.0	2.317871e+10	0.496	0.032	8.495	-1.768	-17.231	...	NaN	0.105	
50188	Zimbabwe	2021	ZWE	15797220.0	2.514009e+10	0.531	0.034	10.204	1.709	20.120	...	NaN	0.104	
50189	Zimbabwe	2022	ZWE	16069061.0	2.590159e+10	0.531	0.033	10.425	0.221	2.169	...	NaN	0.103	
50190	Zimbabwe	2023	ZWE	16340829.0	NaN	0.531	0.032	11.164	0.739	7.090	...	NaN	0.102	

50191 rows × 79 columns

In [6]:

df.columns.to_list()

```
Out[6]: ['country',
        'year',
        'iso_code',
        'population',
        'gdp',
        'cement_co2',
        'cement_co2_per_capita',
        'co2',
        'co2_growth_abs',
        'co2_growth_prct',
        'co2_including_luc',
        'co2_including_luc_growth_abs',
        'co2_including_luc_growth_prct',
        'co2_including_luc_per_capita',
        'co2_including_luc_per_gdp',
        'co2_including_luc_per_unit_energy',
        'co2_per_capita',
        'co2_per_gdp',
        'co2_per_unit_energy',
        'coal_co2',
        'coal_co2_per_capita',
        'consumption_co2',
        'consumption_co2_per_capita',
        'consumption_co2_per_gdp',
        'cumulative_cement_co2',
        'cumulative_co2',
        'cumulative_co2_including_luc',
        'cumulative_coal_co2',
        'cumulative_flaring_co2',
        'cumulative_gas_co2',
        'cumulative_luc_co2',
        'cumulative_oil_co2',
        'cumulative_other_co2',
        'energy_per_capita',
        'energy_per_gdp',
        'flaring_co2',
        'flaring_co2_per_capita',
        'gas_co2',
        'gas_co2_per_capita',
        'ghg_excluding_lucf_per_capita',
        'ghg_per_capita',
        'land_use_change_co2',
        'land_use_change_co2_per_capita',
        'methane',
        'methane_per_capita',
        'nitrous_oxide',
        'nitrous_oxide_per_capita',
        'oil_co2',
        'oil_co2_per_capita',
        'other_co2_per_capita',
        'other_industry_co2',
        'primary_energy_consumption',
        'share_global_cement_co2',
        'share_global_co2',
        'share_global_co2_including_luc',
        'share_global_coal_co2',
```

```
'share_global_cumulative_cement_co2',
'share_global_cumulative_co2',
'share_global_cumulative_co2_including_luc',
'share_global_cumulative_coal_co2',
'share_global_cumulative_flaring_co2',
'share_global_cumulative_gas_co2',
'share_global_cumulative_luc_co2',
'share_global_cumulative_oil_co2',
'share_global_cumulative_other_co2',
'share_global_flaring_co2',
'share_global_gas_co2',
'share_global_luc_co2',
'share_global_oil_co2',
'share_global_other_co2',
'share_of_temperature_change_from_ghg',
'temperature_change_from_ch4',
'temperature_change_from_co2',
'temperature_change_from_ghg',
'temperature_change_from_n2o',
'total_ghg',
'total_ghg_excluding_lucf',
'trade_co2',
'trade_co2_share']
```

```
In [7]: # =====
# Dataset Variable Descriptions (OWID CO2)
# =====

# --- Identifiers ---
# country: Name of the country or region
# year: Year of observation
# iso_code: ISO 3-letter country code (or OWID region code)

# --- Socio-Economic ---
# population: Total population of the country in a given year
# gdp: Gross Domestic Product (GDP), measured in constant international dollars (PPP-adjusted)

# --- Cement Emissions ---
# cement_co2: CO2 emissions from cement production (million tonnes)
# cement_co2_per_capita: CO2 emissions from cement production per person (tonnes)

# --- Core CO2 Emissions ---
# co2: Total CO2 emissions from fossil fuels and cement production (million tonnes)
# co2_per_capita: CO2 emissions per person (tonnes)
# co2_per_gdp: CO2 emissions per unit of GDP (carbon intensity of the economy)
# co2_per_unit_energy: CO2 emissions per unit of energy consumption (carbon intensity of energy supply)

# --- CO2 Growth Metrics ---
# co2_growth_abs: Absolute year-on-year change in CO2 emissions (million tonnes)
# co2_growth_prct: Percentage year-on-year change in CO2 emissions (%)

# --- CO2 Including Land Use Change (LUC) ---
# co2_including_luc: CO2 emissions including land-use change (million tonnes)
# co2_including_luc_per_capita: CO2 emissions including land-use change per person (tonnes)
# co2_including_luc_per_gdp: CO2 emissions including land-use change per unit GDP
# co2_including_luc_per_unit_energy: CO2 emissions including land-use change per unit energy
```

```
# co2_including_luc_growth_abs: Absolute growth in CO2 including land-use change
# co2_including_luc_growth_prct: Percentage growth in CO2 including land-use change

# --- Fossil Fuel-Specific CO2 Emissions ---
# coal_co2: CO2 emissions from coal combustion (million tonnes)
# coal_co2_per_capita: CO2 emissions from coal combustion per person (tonnes)
# oil_co2: CO2 emissions from oil combustion (million tonnes)
# oil_co2_per_capita: CO2 emissions from oil combustion per person (tonnes)
# gas_co2: CO2 emissions from natural gas combustion (million tonnes)
# gas_co2_per_capita: CO2 emissions from gas combustion per person (tonnes)

# --- Flaring & Industrial Emissions ---
# flaring_co2: CO2 emissions from gas flaring during oil and gas production
# flaring_co2_per_capita: CO2 emissions from flaring per person
# other_industry_co2: CO2 emissions from other industrial processes (e.g., chemicals)
# other_co2_per_capita: CO2 emissions from other sources per person

# --- Consumption-Based Emissions ---
# consumption_co2: Consumption-based CO2 emissions (adjusted for trade)
# consumption_co2_per_capita: Consumption-based CO2 emissions per person
# consumption_co2_per_gdp: Consumption-based CO2 emissions per unit GDP

# --- Cumulative Emissions (Historical Totals) ---
# cumulative_co2: Total cumulative CO2 emissions over time
# cumulative_co2_including_luc: Cumulative CO2 emissions including land-use change
# cumulative_cement_co2: Cumulative CO2 emissions from cement
# cumulative_coal_co2: Cumulative CO2 emissions from coal
# cumulative_oil_co2: Cumulative CO2 emissions from oil
# cumulative_gas_co2: Cumulative CO2 emissions from gas
# cumulative_flaring_co2: Cumulative CO2 emissions from flaring
# cumulative_luc_co2: Cumulative CO2 emissions from land-use change
# cumulative_other_co2: Cumulative CO2 emissions from other sources

# --- Energy Indicators ---
# primary_energy_consumption: Total primary energy consumption
# energy_per_capita: Primary energy consumption per person
# energy_per_gdp: Energy intensity of the economy (energy per unit GDP)

# --- Land Use Change Emissions ---
# land_use_change_co2: CO2 emissions from land-use change
# land_use_change_co2_per_capita: CO2 emissions from land-use change per person

# --- Non-CO2 Greenhouse Gases ---
# methane: Methane emissions (CO2-equivalent)
# methane_per_capita: Methane emissions per person
# nitrous_oxide: Nitrous oxide emissions (CO2-equivalent)
# nitrous_oxide_per_capita: Nitrous oxide emissions per person

# --- Total Greenhouse Gas Emissions ---
# total_ghg: Total greenhouse gas emissions (CO2-equivalent)
# total_ghg_excluding_lucf: Total greenhouse gas emissions excluding land-use change and forestry
# ghg_per_capita: Total greenhouse gas emissions per person
# ghg_excluding_lucf_per_capita: GHG emissions per person excluding land-use change and forestry

# --- Global Share Indicators ---
# share_global_co2: Share of global CO2 emissions (%)
```

```
# share_global_co2_including_luc: Share of global CO2 emissions including land-use change
# share_global_cement_co2: Share of global cement-related CO2 emissions
# share_global_coal_co2: Share of global coal-related CO2 emissions
# share_global_oil_co2: Share of global oil-related CO2 emissions
# share_global_gas_co2: Share of global gas-related CO2 emissions
# share_global_flaring_co2: Share of global flaring-related CO2 emissions
# share_global_other_co2: Share of global CO2 from other sources

# --- Global Cumulative Share Indicators ---
# share_global_cumulative_co2: Share of cumulative global CO2 emissions
# share_global_cumulative_co2_including_luc: Share of cumulative CO2 including land-use change
# share_global_cumulative_cement_co2: Share of cumulative cement CO2 emissions
# share_global_cumulative_coal_co2: Share of cumulative coal CO2 emissions
# share_global_cumulative_oil_co2: Share of cumulative oil CO2 emissions
# share_global_cumulative_gas_co2: Share of cumulative gas CO2 emissions
# share_global_cumulative_flaring_co2: Share of cumulative flaring CO2 emissions
# share_global_cumulative_luc_co2: Share of cumulative land-use change CO2 emissions
# share_global_cumulative_other_co2: Share of cumulative other CO2 emissions

# --- Climate Impact Indicators ---
# temperature_change_from_co2: Estimated temperature change attributed to CO2
# temperature_change_from_ch4: Estimated temperature change attributed to methane
# temperature_change_from_n2o: Estimated temperature change attributed to nitrous oxide
# temperature_change_from_ghg: Estimated temperature change from all greenhouse gases
# share_of_temperature_change_from_ghg: Share of global temperature change attributable to GHGs

# --- Trade-Related Emissions ---
# trade_co2: Net CO2 emissions embedded in trade
# trade_co2_share: Share of total CO2 emissions associated with trade
```

In [8]: `df.describe()`

Out[8]:

	year	population	gdp	cement_co2	cement_co2_per_capita	co2	co2_growth_abs	co2_growth_prct	co2_including_luc	co2_including_luc_growth_abs	...	share_global_other_co2	sha
count	50191.000000	4.101900e+04	1.525100e+04	28863.000000	25358.000000	29137.000000	26981.000000	26002.000000	23585.000000	23285.000000	...	2108.000000	
mean	1919.883067	5.686141e+07	3.300794e+11	7.767746	0.059036	415.698178	6.208882	43.104462	535.581202	7.214604	...	7.512655	
std	65.627296	3.199905e+08	3.087720e+12	62.595292	0.120328	1945.843973	62.322553	1729.939596	2202.219657	99.347980	...	17.671054	
min	1750.000000	2.150000e+02	4.998000e+07	0.000000	0.000000	0.000000	-1977.750000	-100.000000	-99.693000	-2325.500000	...	0.000000	
25%	1875.000000	3.273130e+05	7.874038e+09	0.000000	0.000000	0.374000	-0.005000	-1.102500	6.418000	-0.908000	...	0.204750	
50%	1924.000000	2.289522e+06	2.743861e+10	0.000000	0.001000	4.990000	0.044000	3.803500	27.691000	0.078000	...	0.838000	
75%	1974.000000	9.862459e+06	1.210000e+11	0.486000	0.075750	53.273000	1.002000	10.890750	123.959000	2.620000	...	3.211000	
max	2023.000000	8.091735e+09	1.300000e+14	1696.308000	2.484000	37791.570000	1865.208000	180870.000000	41416.480000	2340.184000	...	100.000000	

8 rows × 77 columns



1.2 Handling Missing Values


```
In [10]: # Check total null values in each column
df.isnull().sum()
```

Out[10]: country 0
year 0
iso_code 7929
population 9172
gdp 34940
...
temperature_change_from_n2o 12131
total_ghg 12781
total_ghg_excluding_lucf 12955
trade_co2 45656
trade_co2_share 45656
Length: 79, dtype: int64

```
In [11]: # Check missing values in a clean table format
null_table = df.isnull().sum().reset_index()
null_table.columns = ['Column', 'Missing_Count']

null_table
```

Out[11]:

	Column	Missing_Count
0	country	0
1	year	0
2	iso_code	7929
3	population	9172
4	gdp	34940
...	...	...
74	temperature_change_from_n2o	12131
75	total_ghg	12781
76	total_ghg_excluding_lucf	12955
77	trade_co2	45656
78	trade_co2_share	45656

79 rows × 2 columns

```
In [12]: # Create a tidy summary of missing values
null_summary = pd.DataFrame({
    'Missing_Count': df.isnull().sum(),
    'Missing_Percent (%)': round((df.isnull().sum() / len(df)) * 100, 2)
}).reset_index()

null_summary.columns = ['Column', 'Missing_Count', 'Missing_Percent (%)']
```

null_summary

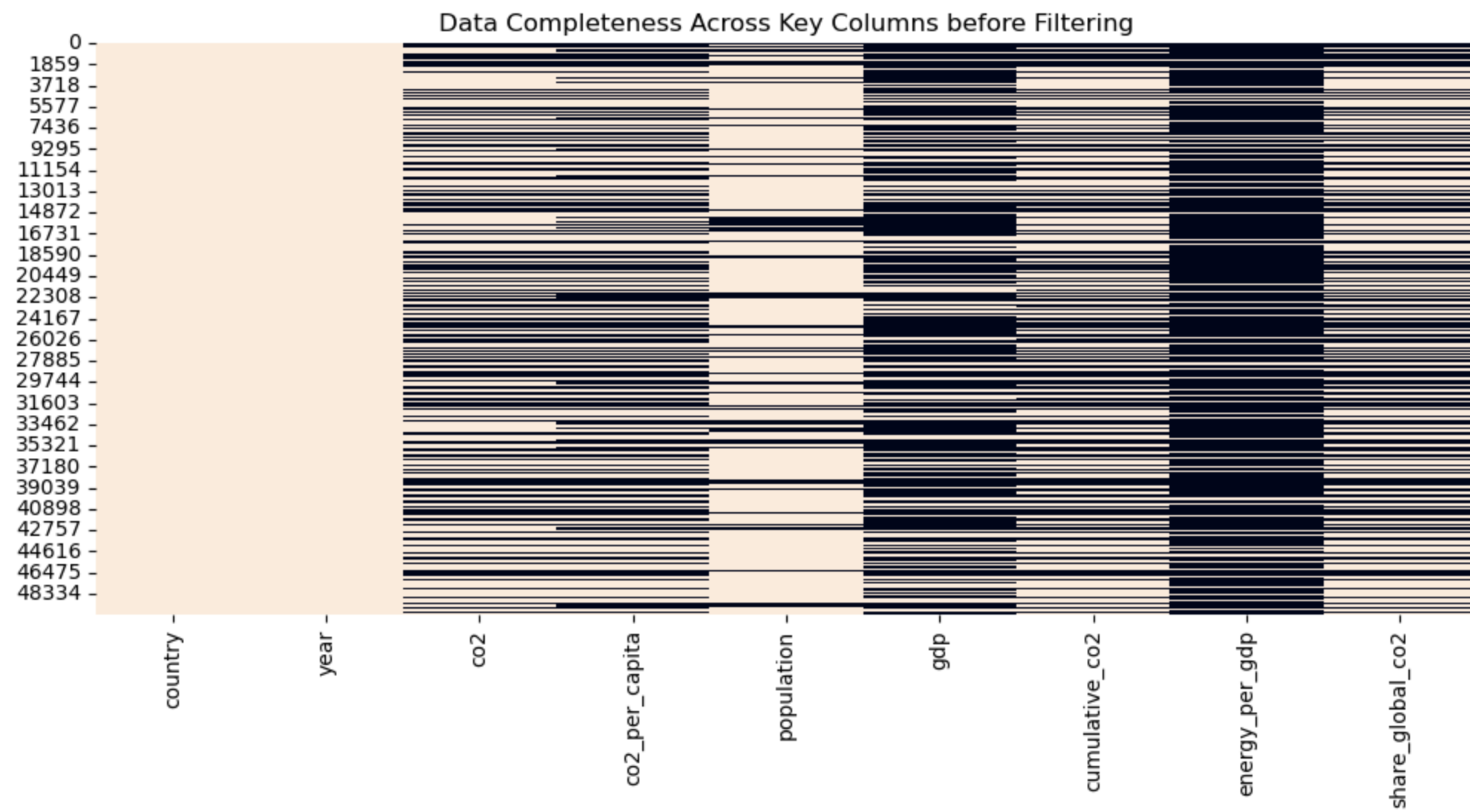
Out[12]:

	Column	Missing_Count	Missing_Percent (%)
0	country	0	0.00
1	year	0	0.00
2	iso_code	7929	15.80
3	population	9172	18.27
4	gdp	34940	69.61
...	...	...	...
74	temperature_change_from_n2o	12131	24.17
75	total_ghg	12781	25.46
76	total_ghg_excluding_lucf	12955	25.81
77	trade_co2	45656	90.96
78	trade_co2_share	45656	90.96

79 rows × 3 columns

Drop dataset before 1990 year

```
In [14]: plt.figure(figsize=(12,5))
sns.heatmap(df[['country','year','co2','co2_per_capita','population','gdp', 'cumulative_co2', 'energy_per_gdp', 'share_global_co2' ]].notnull(), cbar=False)
plt.title("Data Completeness Across Key Columns before Filtering")
plt.show()
```

```
In [15]: # Figure out countries  
df['country'].unique()
```

```
Out[15]: array(['Afghanistan', 'Africa', 'Africa (GCP)', 'Albania', 'Algeria',
               'Andorra', 'Angola', 'Anguilla', 'Antarctica',
               'Antigua and Barbuda', 'Argentina', 'Armenia', 'Aruba', 'Asia',
               'Asia (GCP)', 'Asia (excl. China and India)', 'Australia',
               'Austria', 'Azerbaijan', 'Bahamas', 'Bahrain', 'Bangladesh',
               'Barbados', 'Belarus', 'Belgium', 'Belize', 'Benin', 'Bermuda',
               'Bhutan', 'Bolivia', 'Bonaire Sint Eustatius and Saba',
               'Bosnia and Herzegovina', 'Botswana', 'Brazil',
               'British Virgin Islands', 'Brunei', 'Bulgaria', 'Burkina Faso',
               'Burundi', 'Cambodia', 'Cameroon', 'Canada', 'Cape Verde',
               'Central African Republic', 'Central America (GCP)', 'Chad',
               'Chile', 'China', 'Christmas Island', 'Colombia', 'Comoros',
               'Congo', 'Cook Islands', 'Costa Rica', 'Cote d'Ivoire', 'Croatia',
               'Cuba', 'Curacao', 'Cyprus', 'Czechia',
               'Democratic Republic of Congo', 'Denmark', 'Djibouti', 'Dominica',
               'Dominican Republic', 'East Timor', 'Ecuador', 'Egypt',
               'El Salvador', 'Equatorial Guinea', 'Eritrea', 'Estonia',
               'Eswatini', 'Ethiopia', 'Europe', 'Europe (GCP)',
               'Europe (excl. EU-27)', 'Europe (excl. EU-28)',
               'European Union (27)', 'European Union (28)', 'Faroe Islands',
               'Fiji', 'Finland', 'France', 'French Polynesia', 'Gabon', 'Gambia',
               'Georgia', 'Germany', 'Ghana', 'Greece', 'Greenland', 'Grenada',
               'Guatemala', 'Guinea', 'Guinea-Bissau', 'Guyana', 'Haiti',
               'High-income countries', 'Honduras', 'Hong Kong', 'Hungary',
               'Iceland', 'India', 'Indonesia', 'International aviation',
               'International shipping', 'International transport', 'Iran',
               'Iraq', 'Ireland', 'Israel', 'Italy', 'Jamaica', 'Japan', 'Jordan',
               'Kazakhstan', 'Kenya', 'Kiribati', 'Kosovo', 'Kuwait',
               'Kuwaiti Oil Fires', 'Kuwaiti Oil Fires (GCP)', 'Kyrgyzstan',
               'Laos', 'Latvia', 'Least developed countries (Jones et al.)',
               'Lebanon', 'Lesotho', 'Liberia', 'Libya', 'Liechtenstein',
               'Lithuania', 'Low-income countries',
               'Lower-middle-income countries', 'Luxembourg', 'Macao',
               'Madagascar', 'Malawi', 'Malaysia', 'Maldives', 'Mali', 'Malta',
               'Marshall Islands', 'Mauritania', 'Mauritius', 'Mexico',
               'Micronesia (country)', 'Middle East (GCP)', 'Moldova', 'Monaco',
               'Mongolia', 'Montenegro', 'Montserrat', 'Morocco', 'Mozambique',
               'Myanmar', 'Namibia', 'Nauru', 'Nepal', 'Netherlands',
               'New Caledonia', 'New Zealand', 'Nicaragua', 'Niger', 'Nigeria',
               'Niue', 'Non-OECD (GCP)', 'North America', 'North America (GCP)',
               'North America (excl. USA)', 'North Korea', 'North Macedonia',
               'Norway', 'OECD (GCP)', 'OECD (Jones et al.)', 'Oceania',
               'Oceania (GCP)', 'Oman', 'Pakistan', 'Palau', 'Palestine',
               'Panama', 'Papua New Guinea', 'Paraguay', 'Peru', 'Philippines',
               'Poland', 'Portugal', 'Qatar', 'Romania', 'Russia', 'Rwanda',
               'Ryukyu Islands', 'Ryukyu Islands (GCP)', 'Saint Helena',
               'Saint Kitts and Nevis', 'Saint Lucia',
               'Saint Pierre and Miquelon', 'Saint Vincent and the Grenadines',
               'Samoa', 'San Marino', 'Sao Tome and Principe', 'Saudi Arabia',
               'Senegal', 'Serbia', 'Seychelles', 'Sierra Leone', 'Singapore',
               'Sint Maarten (Dutch part)', 'Slovakia', 'Slovenia',
               'Solomon Islands', 'Somalia', 'South Africa', 'South America',
               'South America (GCP)', 'South Korea', 'South Sudan', 'Spain',
               'Sri Lanka', 'Sudan', 'Suriname', 'Sweden', 'Switzerland', 'Syria',
               'Taiwan', 'Tajikistan', 'Tanzania', 'Thailand', 'Togo', 'Tonga',
               'Trinidad and Tobago', 'Tunisia', 'Turkey', 'Turkmenistan',
```

```
'Turks and Caicos Islands', 'Tuvalu', 'Uganda', 'Ukraine',  
'United Arab Emirates', 'United Kingdom', 'United States',  
'Upper-middle-income countries', 'Uruguay', 'Uzbekistan',  
'Vanuatu', 'Vatican', 'Venezuela', 'Vietnam', 'Wallis and Futuna',  
'World', 'Yemen', 'Zambia', 'Zimbabwe'], dtype=object)
```

There are a lot of duplicate country names so we will remove unnecessary countries by using iso code.

Filtering by year and country ISO code

```
In [18]: before = len(df)  
print(f"Row Before filtering: {before}")
```

Row Before filtering: 50191

```
In [19]: # Update dataframe into only updated year and country with ISO code  
  
# Keep only years from 1990 onwards  
# Reason: Before 1990, most selected socio-economic and sector-based emission variables have many null values.  
  
# Filter to ensure it's a real country (which has an ISO code)  
df = df[(df['year'] >= 1990) & (df['iso_code'].notnull())]
```

```
In [20]: # Count rows after filtering  
after = len(df)  
print(after)
```

7412

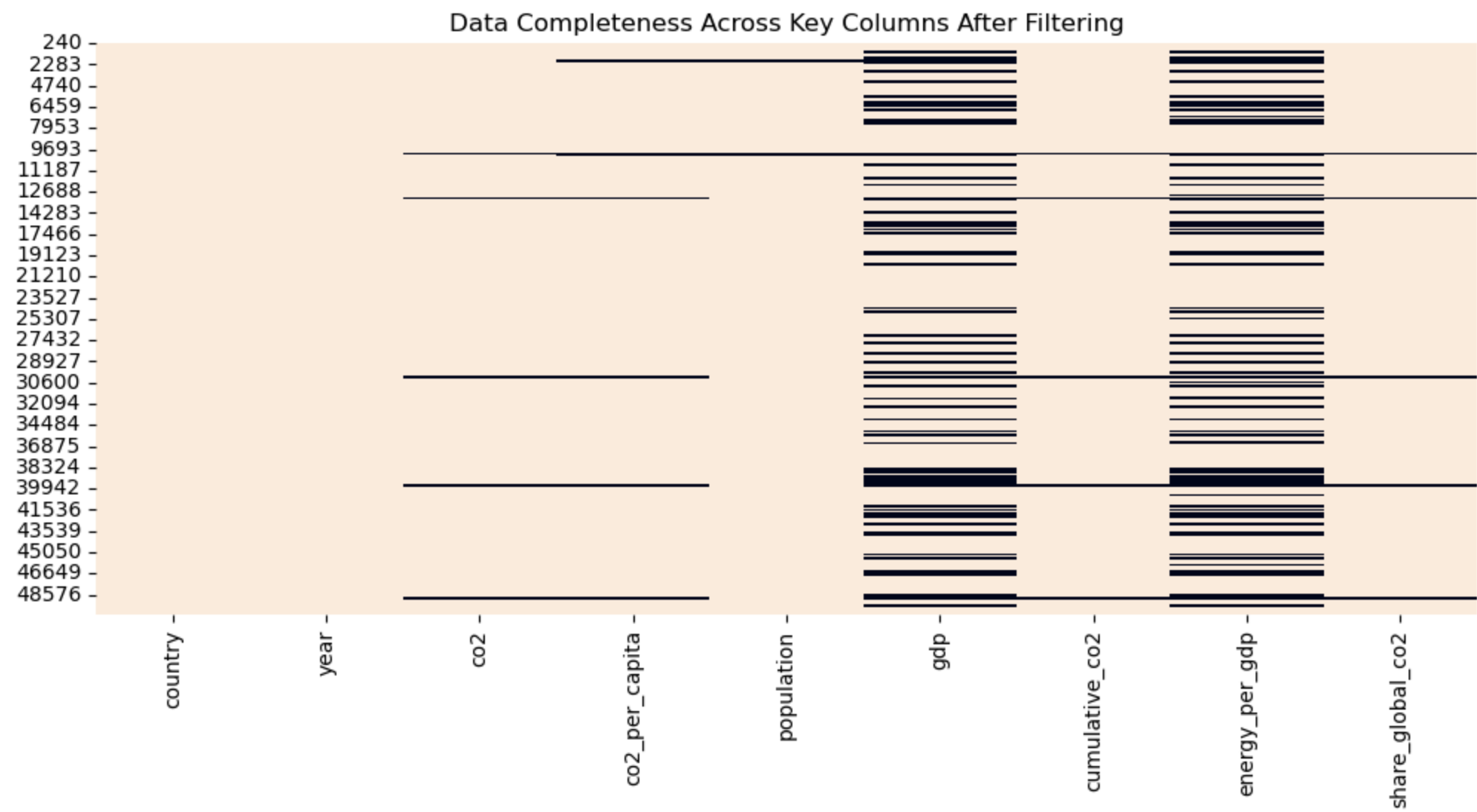
```
In [21]: # Show how many rows were removed  
print("Rows before:", before)  
print("Rows after :", after)  
print("Removed    :", before - after)
```

Rows before: 50191

Rows after : 7412

Removed : 42779

```
In [22]: plt.figure(figsize=(12,5))  
sns.heatmap(df[['country','year','co2','co2_per_capita','population','gdp', 'cumulative_co2', 'energy_per_gdp', 'share_global_co2' ]].notnull(), cbar=False)  
plt.title("Data Completeness Across Key Columns After Filtering")  
plt.show()
```



```
In [23]: df.head(10)
```

Out[23]:

	country	year	iso_code	population	gdp	cement_co2	cement_co2_per_capita	co2	co2_growth_abs	co2_growth_prct	...	share_global_other_co2	share_of_temperature_change_from_ghg	temperatu
240	Afghanistan	1990	AFG	12045664.0	1.306598e+10	0.046	0.004	2.024	-0.741	-26.784	...	NaN	0.094	
241	Afghanistan	1991	AFG	12238879.0	1.204736e+10	0.046	0.004	1.914	-0.110	-5.435	...	NaN	0.092	
242	Afghanistan	1992	AFG	13278982.0	1.267754e+10	0.046	0.003	1.482	-0.432	-22.580	...	NaN	0.090	
243	Afghanistan	1993	AFG	14943174.0	9.834582e+09	0.047	0.003	1.487	0.005	0.330	...	NaN	0.089	
244	Afghanistan	1994	AFG	16250799.0	7.919857e+09	0.047	0.003	1.454	-0.033	-2.227	...	NaN	0.087	
245	Afghanistan	1995	AFG	17065836.0	1.230753e+10	0.047	0.003	1.417	-0.037	-2.511	...	NaN	0.085	
246	Afghanistan	1996	AFG	17763265.0	1.207013e+10	0.047	0.003	1.370	-0.047	-3.332	...	NaN	0.084	
247	Afghanistan	1997	AFG	18452100.0	1.185075e+10	0.047	0.003	1.304	-0.066	-4.814	...	NaN	0.083	
248	Afghanistan	1998	AFG	19159996.0	1.169217e+10	0.047	0.002	1.279	-0.026	-1.967	...	NaN	0.082	
249	Afghanistan	1999	AFG	19887791.0	1.151732e+10	0.047	0.002	1.092	-0.187	-14.616	...	NaN	0.082	

10 rows × 79 columns



In [24]:

```
df.tail(10)
```

Out[24]:

	country	year	iso_code	population	gdp	cement_co2	cement_co2_per_capita	co2	co2_growth_abs	co2_growth_prct	...	share_global_other_co2	share_of_temperature_change_from_ghg	tempera
50181	Zimbabwe	2014	ZWE	14207367.0	2.381884e+10	0.496	0.035	11.946	0.275	2.354	...	NaN	0.110	
50182	Zimbabwe	2015	ZWE	14399009.0	2.424616e+10	0.585	0.041	12.017	0.071	0.591	...	NaN	0.109	
50183	Zimbabwe	2016	ZWE	14600297.0	2.436205e+10	0.461	0.032	10.581	-1.436	-11.948	...	NaN	0.109	
50184	Zimbabwe	2017	ZWE	14812484.0	2.557601e+10	0.469	0.032	9.827	-0.754	-7.129	...	NaN	0.108	
50185	Zimbabwe	2018	ZWE	15034457.0	2.678627e+10	0.558	0.037	11.205	1.379	14.030	...	NaN	0.107	
50186	Zimbabwe	2019	ZWE	15271377.0	2.514642e+10	0.473	0.031	10.263	-0.942	-8.411	...	NaN	0.106	
50187	Zimbabwe	2020	ZWE	15526888.0	2.317871e+10	0.496	0.032	8.495	-1.768	-17.231	...	NaN	0.105	
50188	Zimbabwe	2021	ZWE	15797220.0	2.514009e+10	0.531	0.034	10.204	1.709	20.120	...	NaN	0.104	
50189	Zimbabwe	2022	ZWE	16069061.0	2.590159e+10	0.531	0.033	10.425	0.221	2.169	...	NaN	0.103	
50190	Zimbabwe	2023	ZWE	16340829.0	NaN	0.531	0.032	11.164	0.739	7.090	...	NaN	0.102	

10 rows × 79 columns



Descriptive Analytics

```
In [26]: df.shape
```

Out[26]: (7412, 79)

```
In [27]: df.dtypes
```

Out[27]: country object
year int64
iso_code object
population float64
gdp float64

...
temperature_change_from_n2o float64
total_ghg float64
total_ghg_excluding_lucf float64
trade_co2 float64
trade_co2_share float64
Length: 79, dtype: object

```
In [28]: df[['co2', 'co2_per_capita', 'population', 'gdp']].describe().T
```

Out[28]:

	count	mean	std	min	25%	50%	75%	max
co2	7293.0	1.364462e+02	6.654744e+02	0.0	8.060000e-01	7.015000e+00	5.395200e+01	1.190250e+04
co2_per_capita	7229.0	5.026050e+00	7.839179e+00	0.0	7.070000e-01	2.729000e+00	6.858000e+00	3.646880e+02
population	7344.0	3.111503e+07	1.247305e+08	501.0	6.335550e+05	5.496358e+06	1.982209e+07	1.438070e+09
gdp	5410.0	4.894207e+11	1.704855e+12	257172000.0	1.885143e+10	6.108450e+10	2.910000e+11	2.700000e+13

```
In [29]: df.nunique()
```

Out[29]: country 218
year 34
iso_code 218
population 7328
gdp 4064

...
temperature_change_from_n2o 12
total_ghg 6204
total_ghg_excluding_lucf 5987
trade_co2 3638
trade_co2_share 3820
Length: 79, dtype: int64

```
In [30]: #CO2 emission trends  
df['co2'].describe()
```



```
Out[30]: count      7293.000000  
         mean       136.446164  
         std        665.474372  
         min         0.000000  
         25%         0.806000  
         50%         7.015000  
         75%        53.952000  
         max       11902.503000  
         Name: co2, dtype: float64
```

```
In [31]: df['country'].unique()
```

```
Out[31]: array(['Afghanistan', 'Albania', 'Algeria', 'Andorra', 'Angola',
               'Anguilla', 'Antarctica', 'Antigua and Barbuda', 'Argentina',
               'Armenia', 'Aruba', 'Australia', 'Austria', 'Azerbaijan',
               'Bahamas', 'Bahrain', 'Bangladesh', 'Barbados', 'Belarus',
               'Belgium', 'Belize', 'Benin', 'Bermuda', 'Bhutan', 'Bolivia',
               'Bonaire Sint Eustatius and Saba', 'Bosnia and Herzegovina',
               'Botswana', 'Brazil', 'British Virgin Islands', 'Brunei',
               'Bulgaria', 'Burkina Faso', 'Burundi', 'Cambodia', 'Cameroon',
               'Canada', 'Cape Verde', 'Central African Republic', 'Chad',
               'Chile', 'China', 'Christmas Island', 'Colombia', 'Comoros',
               'Congo', 'Cook Islands', 'Costa Rica', 'Cote d'Ivoire', 'Croatia',
               'Cuba', 'Curacao', 'Cyprus', 'Czechia',
               'Democratic Republic of Congo', 'Denmark', 'Djibouti', 'Dominica',
               'Dominican Republic', 'East Timor', 'Ecuador', 'Egypt',
               'El Salvador', 'Equatorial Guinea', 'Eritrea', 'Estonia',
               'Eswatini', 'Ethiopia', 'Faroe Islands', 'Fiji', 'Finland',
               'France', 'French Polynesia', 'Gabon', 'Gambia', 'Georgia',
               'Germany', 'Ghana', 'Greece', 'Greenland', 'Grenada', 'Guatemala',
               'Guinea', 'Guinea-Bissau', 'Guyana', 'Haiti', 'Honduras',
               'Hong Kong', 'Hungary', 'Iceland', 'India', 'Indonesia', 'Iran',
               'Iraq', 'Ireland', 'Israel', 'Italy', 'Jamaica', 'Japan', 'Jordan',
               'Kazakhstan', 'Kenya', 'Kiribati', 'Kuwait', 'Kyrgyzstan', 'Laos',
               'Latvia', 'Lebanon', 'Lesotho', 'Liberia', 'Libya',
               'Liechtenstein', 'Lithuania', 'Luxembourg', 'Macao', 'Madagascar',
               'Malawi', 'Malaysia', 'Maldives', 'Mali', 'Malta',
               'Marshall Islands', 'Mauritania', 'Mauritius', 'Mexico',
               'Micronesia (country)', 'Moldova', 'Monaco', 'Mongolia',
               'Montenegro', 'Montserrat', 'Morocco', 'Mozambique', 'Myanmar',
               'Namibia', 'Nauru', 'Nepal', 'Netherlands', 'New Caledonia',
               'New Zealand', 'Nicaragua', 'Niger', 'Nigeria', 'Niue',
               'North Korea', 'North Macedonia', 'Norway', 'Oman', 'Pakistan',
               'Palau', 'Palestine', 'Panama', 'Papua New Guinea', 'Paraguay',
               'Peru', 'Philippines', 'Poland', 'Portugal', 'Qatar', 'Romania',
               'Russia', 'Rwanda', 'Saint Helena', 'Saint Kitts and Nevis',
               'Saint Lucia', 'Saint Pierre and Miquelon',
               'Saint Vincent and the Grenadines', 'Samoa', 'San Marino',
               'Sao Tome and Principe', 'Saudi Arabia', 'Senegal', 'Serbia',
               'Seychelles', 'Sierra Leone', 'Singapore',
               'Sint Maarten (Dutch part)', 'Slovakia', 'Slovenia',
               'Solomon Islands', 'Somalia', 'South Africa', 'South Korea',
               'South Sudan', 'Spain', 'Sri Lanka', 'Sudan', 'Suriname', 'Sweden',
               'Switzerland', 'Syria', 'Taiwan', 'Tajikistan', 'Tanzania',
               'Thailand', 'Togo', 'Tonga', 'Trinidad and Tobago', 'Tunisia',
               'Turkey', 'Turkmenistan', 'Turks and Caicos Islands', 'Tuvalu',
               'Uganda', 'Ukraine', 'United Arab Emirates', 'United Kingdom',
               'United States', 'Uruguay', 'Uzbekistan', 'Vanuatu', 'Vatican',
               'Venezuela', 'Vietnam', 'Wallis and Futuna', 'Yemen', 'Zambia',
               'Zimbabwe'], dtype=object)
```

```
In [32]: # 2.2 Outlier Detection (Boxplots)
# This helps identify if specific years or countries are statistical anomalies
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='year', y='co2_per_capita', palette="viridis")
plt.xticks(rotation=90) # Rotate for readability
```

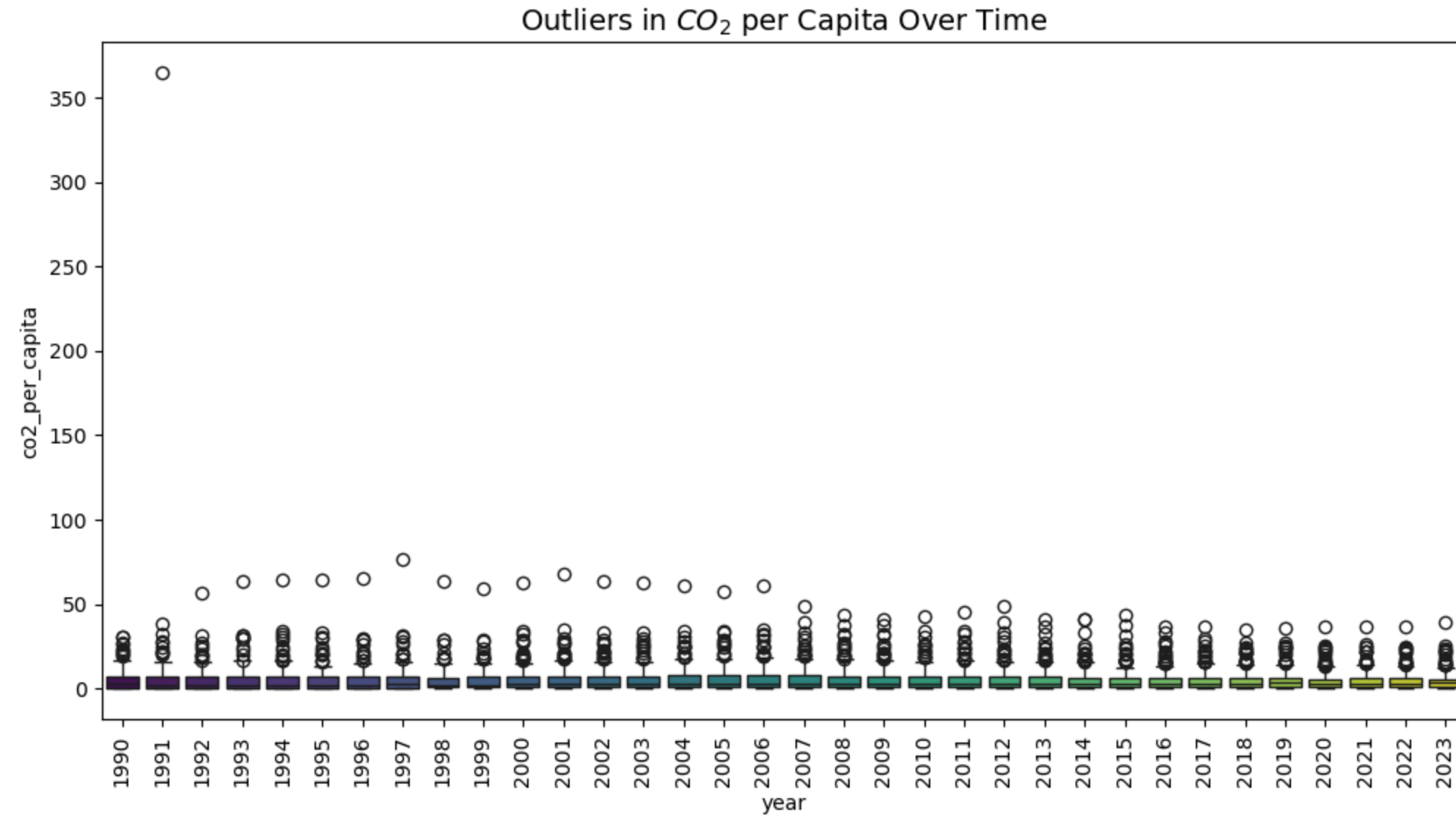


```
plt.title('Outliers in $CO_2$ per Capita Over Time', fontsize=14)
plt.show()
```

C:\Users\USER\AppData\Local\Temp\ipykernel_5704\3438707602.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='year', y='co2_per_capita', palette="viridis")
```



```
In [33]: df['log_co2_per_capita'] = np.log1p(df['co2_per_capita'])

# Transformed Outlier Detection (Boxplots)
plt.figure(figsize=(14, 7))

# We use the new Log-transformed column for the y-axis
sns.boxplot(data=df, x='year', y='log_co2_per_capita', palette="viridis", fliersize=2)

# Aesthetics and Labels
plt.xticks(rotation=90, fontsize=8)
plt.title('Log-Transformed $CO_2$ per Capita Over Time (Reduced Skewness)', fontsize=16)
plt.ylabel('Log($CO_2$ per Capita + 1)', fontsize=12)
plt.xlabel('Year', fontsize=12)
```

```
# Optional: Add a subtle grid for better alignment
plt.grid(axis='y', linestyle='--', alpha=0.4)

sns.despine() # Clean up the chart edges
plt.tight_layout()
plt.show()
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_5704\1404250965.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

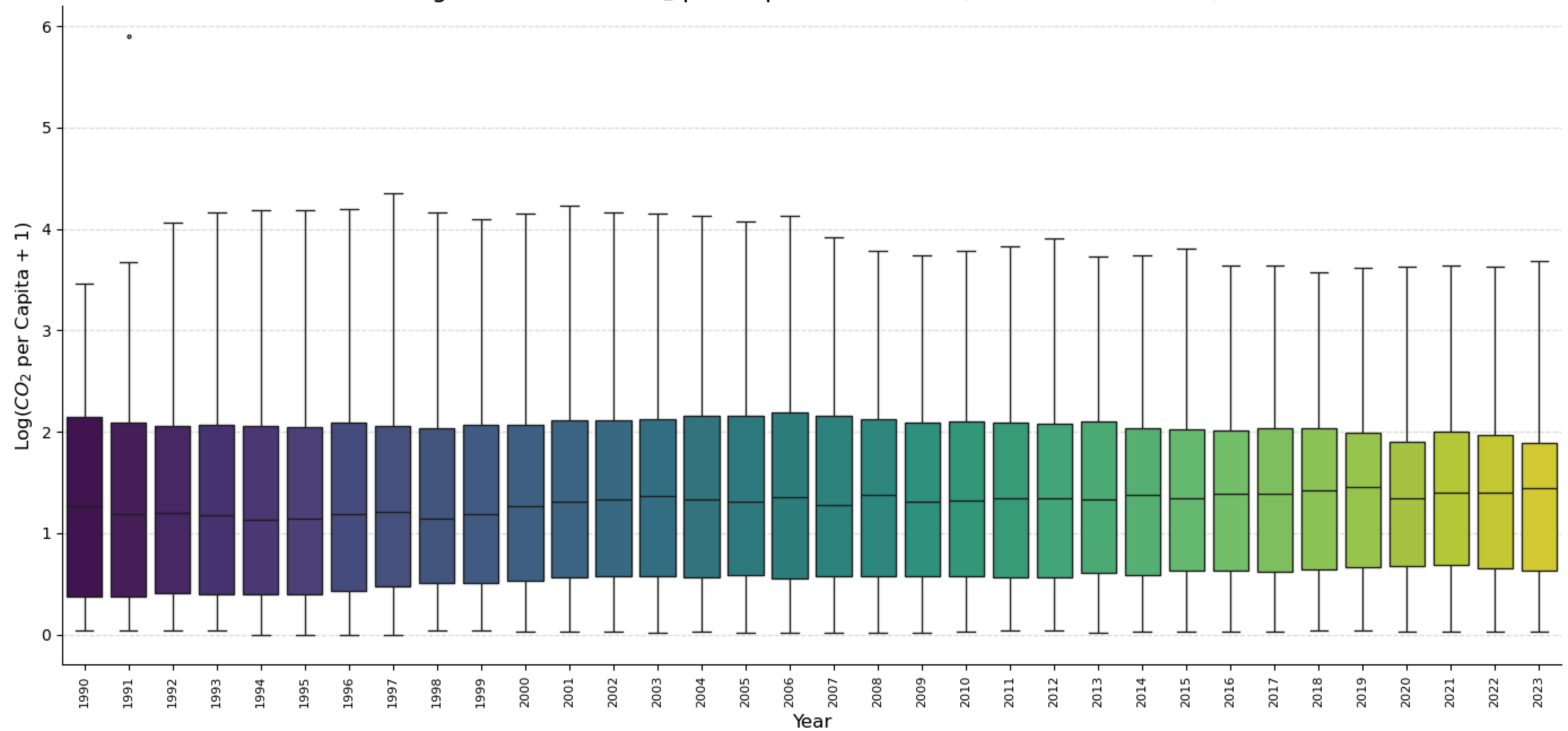
```
df['log_co2_per_capita'] = np.log1p(df['co2_per_capita'])
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_5704\1404250965.py:7: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='year', y='log_co2_per_capita', palette="viridis", fliersize=2)
```

Log-Transformed CO₂ per Capita Over Time (Reduced Skewness)



```
In [34]: # Identify top 10 emitters based on the most recent data (or max historical)
# Using max() captures the peak emitters, which is standard for your RQs
top_10_countries = (
    df.groupby('country')['co2']
      .max()
      .sort_values(ascending=False)
      .head(10)
      .index.tolist()
)
print(f"Top 10 Emitters Identified: {top_10_countries}")
```

Top 10 Emitters Identified: ['China', 'United States', 'India', 'Russia', 'Japan', 'Germany', 'Iran', 'Indonesia', 'Saudi Arabia', 'Ukraine']

```
In [35]: plt.figure(figsize=(14, 8))
```

```
# We use the original 'co2' column but apply the log scale via Matplotlib
# This keeps the Y-axis labels in 'Million Tonnes' but spaced logarithmically
sns.lineplot(
    data=df[df['country'].isin(top_10_countries)],
    x='year',
    y='co2',
    hue='country',
    linewidth=2.5,
    marker='o',          # Adding markers helps distinguish points on a log scale
    markersize=4
)

# 1. Apply Logarithmic Scale to the Y-axis
plt.yscale('log')

# 2. Set the Title using LaTeX formatting
plt.title(r'Log-Scaled Historical $CO_2$ Trends for Top 10 Global Emitters (1990-2022)', fontsize=16)

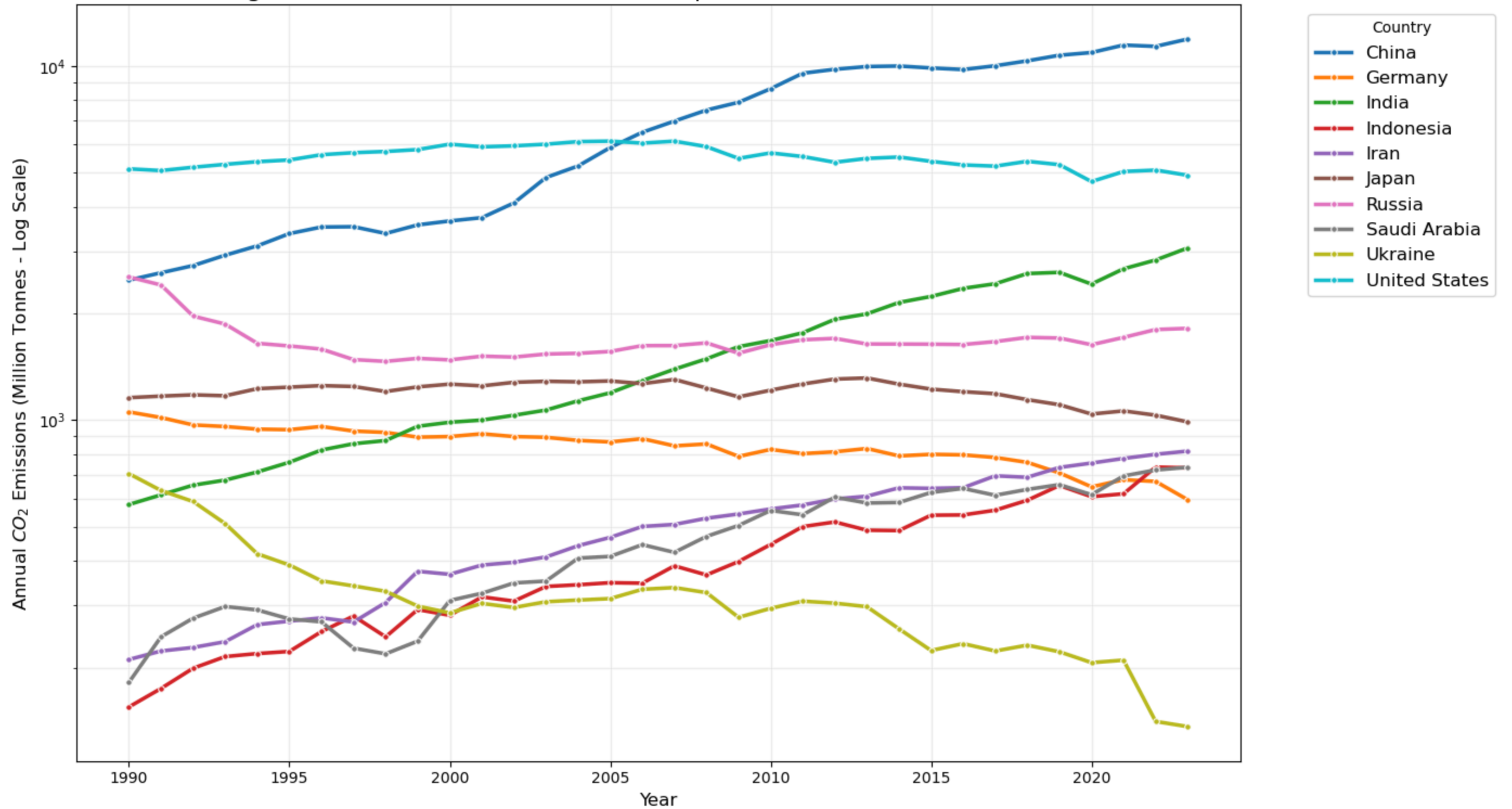
# 3. Enhance Labels
plt.ylabel(r'Annual $CO_2$ Emissions (Million Tonnes - Log Scale)', fontsize=12)
plt.xlabel('Year', fontsize=12)

# 4. Improve Legend Placement
plt.legend(title='Country', bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=12)

# 5. Add Gridlines (very helpful for reading log charts)
plt.grid(True, which="both", ls="--", alpha=0.2)

plt.tight_layout()
plt.show()
```

Log-Scaled Historical CO₂ Trends for Top 10 Global Emitters (1990-2022)



```
In [36]: # 1. Prepare the data for the most recent year
latest_year = df['year'].max()
df_map = df[df['year'] == latest_year].copy()

# 2. Create a Logarithmic version of CO2 for the color scale
# This prevents the 'Super-Emitters' from washing out the map
df_map['log_co2'] = np.log10(df_map['co2'] + 1)
```

```

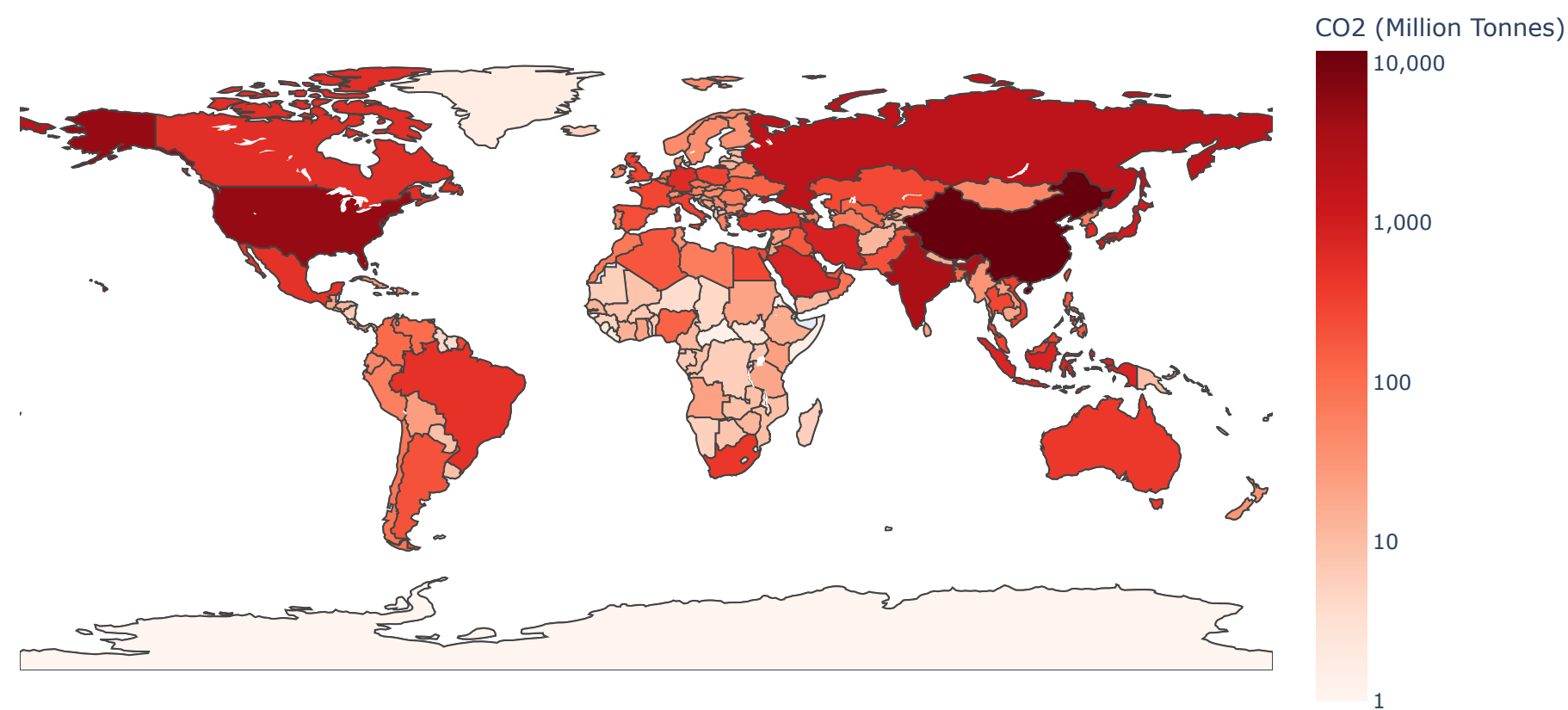
# 3. Create the Map
fig = px.choropleth(
    df_map,
    locations="country",
    locationmode='country names',
    color="log_co2",          # Color based on log scale
    hover_name="country",
    hover_data={"co2": True, "log_co2": False}, # Show real CO2 value on hover
    color_continuous_scale=px.colors.sequential.Reds,
    title=f"Global CO2 Emissions Distribution ({latest_year})",
    labels={'log_co2': 'Emissions Intensity (Log Scale)'}
)

# 4. Clean up the layout
fig.update_layout(
    geo=dict(
        showframe=False,
        showcoastlines=True,
        projection_type='equirectangular'
    ),
    width=1000,
    height=600,
    coloraxis_colorbar=dict(
        title="CO2 (Million Tonnes)",
        tickvals=[0, 1, 2, 3, 4, 5, 6, 7, 8],
        ticktext=['1', '10', '100', '1,000', '10,000'] # Mapping log back to real numbers
    )
)

fig.show()

```


Global CO2 Emissions Distribution (2023)



```
In [37]: # Professional Styling
plt.style.use('seaborn-v0_8-white') # Clean, modern look
fig, ax1 = plt.subplots(figsize=(12, 6), dpi=100)

# Filter data
comparison = df[df['country'].isin(['United Kingdom', 'India'])]

# 1. Plot GDP on the Primary Axis (Dashed)
gdp_plot = sns.lineplot(data=comparison, x='year', y='gdp', hue='country',
                        ax=ax1, linestyle='--', legend=False, palette=['#1f77b4', '#ff7f0e'])
ax1.set_ylabel('GDP (PPP-Adjusted, Billions $)', fontsize=12, fontweight='bold')
ax1.grid(True, linestyle=':', alpha=0.6)

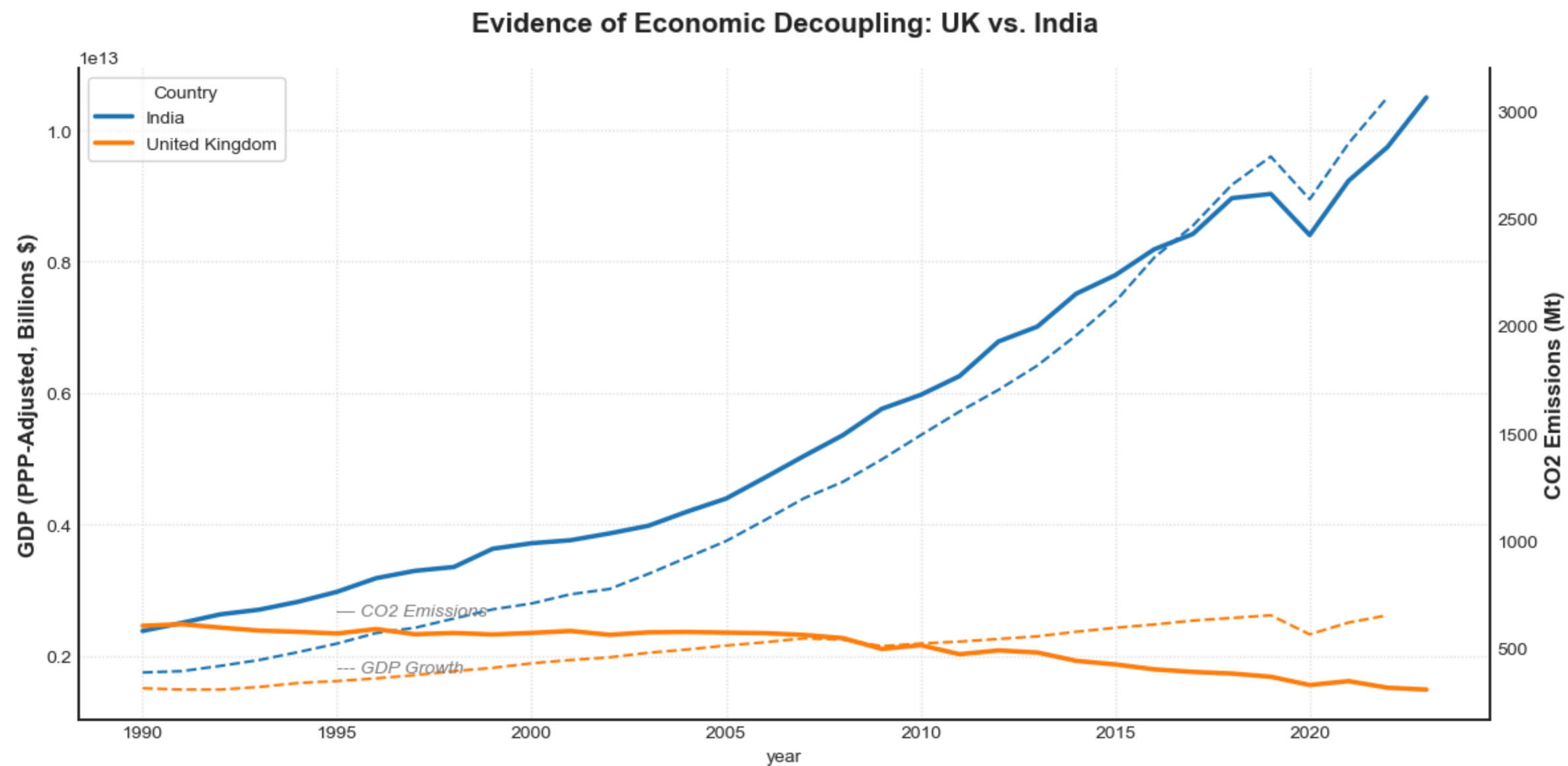
# 2. Plot CO2 on the Secondary Axis (Solid)
ax2 = ax1.twinx()
co2_plot = sns.lineplot(data=comparison, x='year', y='co2', hue='country',
                        ax=ax2, palette=['#1f77b4', '#ff7f0e'], linewidth=2.5)
ax2.set_ylabel('CO2 Emissions (Mt)', fontsize=12, fontweight='bold')
```

```
# 3. Refined Titles and Spines
plt.title('Evidence of Economic Decoupling: UK vs. India', fontsize=15, pad=20, fontweight='bold')
ax1.spines['top'].set_visible(False)
ax2.spines['top'].set_visible(False)

# 4. Better Legend Handling
ax2.legend(title='Country', frameon=True, loc='upper left')

# Text annotations for clarity
ax1.text(1995, comparison[comparison['country']=='India']['gdp'].min(),
        "--- GDP Growth", color='grey', fontsize=10, style='italic')
ax1.text(1995, comparison[comparison['country']=='India']['gdp'].min() * 1.5,
        "- CO2 Emissions", color='grey', fontsize=10, style='italic')

plt.tight_layout()
plt.show()
```



```
In [38]: # Set a clean, professional background style
sns.set_style("whitegrid", {'axes.grid': False})
plt.figure(figsize=(10, 6), dpi=100)
```



```

# Create the regression plot
# We use 'regplot' but customize it to look like a high-end journal figure
ax = sns.regplot(
    data=df,
    x='gdp',
    y='co2',
    scatter_kws={'alpha':0.2, 'color':'#2b5a7a', 's':25}, # Smaller, subtle navy points
    line_kws={'color':'#d9534f', 'lw':2},                # Clean, muted red for the trend
    label='Global Correlation'
)

# 1. Apply Log Scales
plt.xscale('log')
plt.yscale('log')

# 2. Add Titles and Labels with proper formatting
plt.title('The Global "Engine": Economic Scale vs. Carbon Output',
          fontsize=16, fontweight='bold', pad=20, loc='left')
plt.xlabel('Gross Domestic Product (PPP-Adjusted, Log Scale)', fontsize=11)
plt.ylabel('Annual $CO_2$ Emissions (Million Tonnes, Log Scale)', fontsize=11)

# 3. Aesthetics: Remove top and right spines for a modern look
sns.despine()

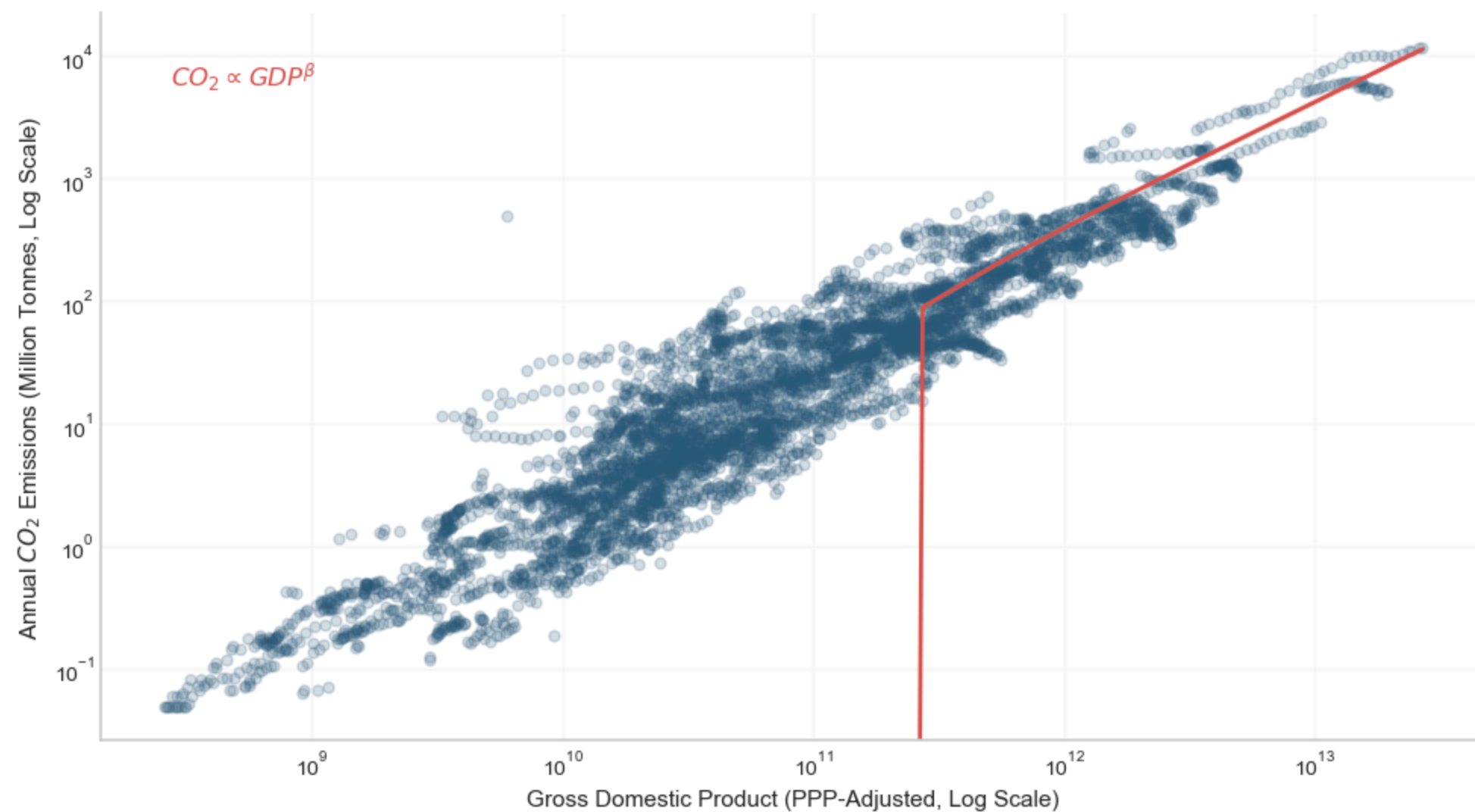
# 4. Add a soft grid only for the major log-lines to help readability
plt.grid(True, which="major", ls="-", alpha=0.15)

# 5. Add a mathematical annotation (Optional but very professional)
# This explains that on a log-log plot, the line represents a power law
plt.text(0.05, 0.90, r'$CO_2 \propto GDP^{\beta}$',
         transform=ax.transAxes, fontsize=12, color='#d9534f', fontweight='bold')

plt.tight_layout()
plt.show()

```

The Global "Engine": Economic Scale vs. Carbon Output



Feature Selection

We Select feature by using pearson Correlation matrix.

```
In [40]: # 1. Calculate the correlation matrix
corr_matrix = df.corr(numeric_only=True)

# 2. Focus on Correlation with Target (co2)
# We want features that have a high absolute correlation with our target
target_corr = corr_matrix['co2'].sort_values(ascending=False)

# Selection Threshold: Typically features with |r| > 0.5 are considered strong
threshold = 0.5
selected_features = target_corr[abs(target_corr) > threshold].index.tolist()

print(f"Features highly correlated with CO2 (>{threshold}):\n", target_corr[abs(target_corr) > threshold])
```

```
Features highly correlated with CO2 (>0.5):
  co2      1.000000
total_ghg_excluding_lucf  0.998909
consumption_co2          0.993456
primary_energy_consumption 0.987945
share_global_co2         0.984704
co2_including_luc        0.980729
total_ghg                 0.975385
share_global_co2_including_luc 0.964241
gdp                       0.946136
share_global_cumulative_cement_co2 0.941300
share_global_coal_co2     0.936523
coal_co2                  0.936229
cumulative_other_co2      0.927056
cumulative_cement_co2     0.903707
other_industry_co2        0.892762
cumulative_coal_co2       0.889078
temperature_change_from_ch4 0.884026
nitrous_oxide             0.878688
share_global_other_co2    0.876175
methane                   0.872701
temperature_change_from_ghg 0.870564
share_global_cumulative_coal_co2 0.865389
oil_co2                   0.864133
share_global_cumulative_other_co2 0.860313
temperature_change_from_n2o 0.858272
share_of_temperature_change_from_ghg 0.853050
share_global_oil_co2      0.848962
cumulative_co2            0.835109
cement_co2                0.834186
cumulative_co2_including_luc 0.833653
share_global_cement_co2   0.830113
temperature_change_from_co2 0.827081
share_global_cumulative_co2_including_luc 0.816815
share_global_cumulative_co2 0.814472
population                0.729271
gas_co2                   0.708480
cumulative_oil_co2        0.705865
share_global_cumulative_oil_co2 0.702305
share_global_gas_co2      0.685203
cumulative_luc_co2        0.665673
share_global_cumulative_luc_co2 0.659778
share_global_cumulative_gas_co2 0.609166
cumulative_gas_co2        0.602581
Name: co2, dtype: float64
```

```
In [41]: # =====
# Final Feature Selection for RQ1 - RQ4
# =====

# 1. Identifiers
identifiers = ['country', 'year']

# 2. Target Variable (For RQ2 Forecasting)
target = ['co2']
```

```

# 3. Socio-Economic & Energy Drivers (For RQ2 Forecasting & RQ3 Explainability)
# These represent the "Scale" and "Demand" of an economy
drivers = [
    'gdp',
    'population',
    'primary_energy_consumption'
]

# 4. Behavioral & Intensity Indicators (For RQ1 Clustering)
# These allow us to compare a small green country vs a small coal-dependent country fairly
behavioral_metrics = [
    'co2_per_capita',      # Standard of living vs emissions
    'energy_per_gdp',      # Energy efficiency of the economy
    'co2_per_gdp',         # Carbon intensity of the economy
    'coal_co2_per_capita', # Dependence on coal
    'oil_co2_per_capita',  # Dependence on oil
    'gas_co2_per_capita'   # Dependence on gas
]

# Combine into one master list
all_selected_cols = identifiers + target + drivers + behavioral_metrics

# Filter the engineered dataframe
df_final = df[all_selected_cols].copy()

print(f"Dataset prepared with {len(all_selected_cols)} features.")
print(df_final.columns.tolist())

```

Dataset prepared with 12 features.

```
['country', 'year', 'co2', 'gdp', 'population', 'primary_energy_consumption', 'co2_per_capita', 'energy_per_gdp', 'co2_per_gdp', 'coal_co2_per_capita', 'oil_co2_per_capita', 'gas_co2_per_capita']
```

```

In [42]: # Define the final feature list (based on selection)
selected_features = [
    'year', 'co2', 'gdp', 'population', 'primary_energy_consumption',
    'co2_per_capita', 'energy_per_gdp', 'co2_per_gdp',
    'coal_co2_per_capita', 'oil_co2_per_capita', 'gas_co2_per_capita'
]

# Filter the dataframe and calculate the correlation matrix
# Note: 'country' is excluded automatically as it is non-numeric
corr_matrix = df[selected_features].corr()

# Visualization Setup
sns.set_theme(style="white")
plt.figure(figsize=(12, 10))

# Create a mask to hide the upper triangle (removes redundancy)
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# 4. Plotting the Heatmap
heatmap = sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,          # Show correlation values
    fmt=".2f",           # 2 decimal places
    cmap='RdBu_r',       # Red-Blue palette (standard for correlation)

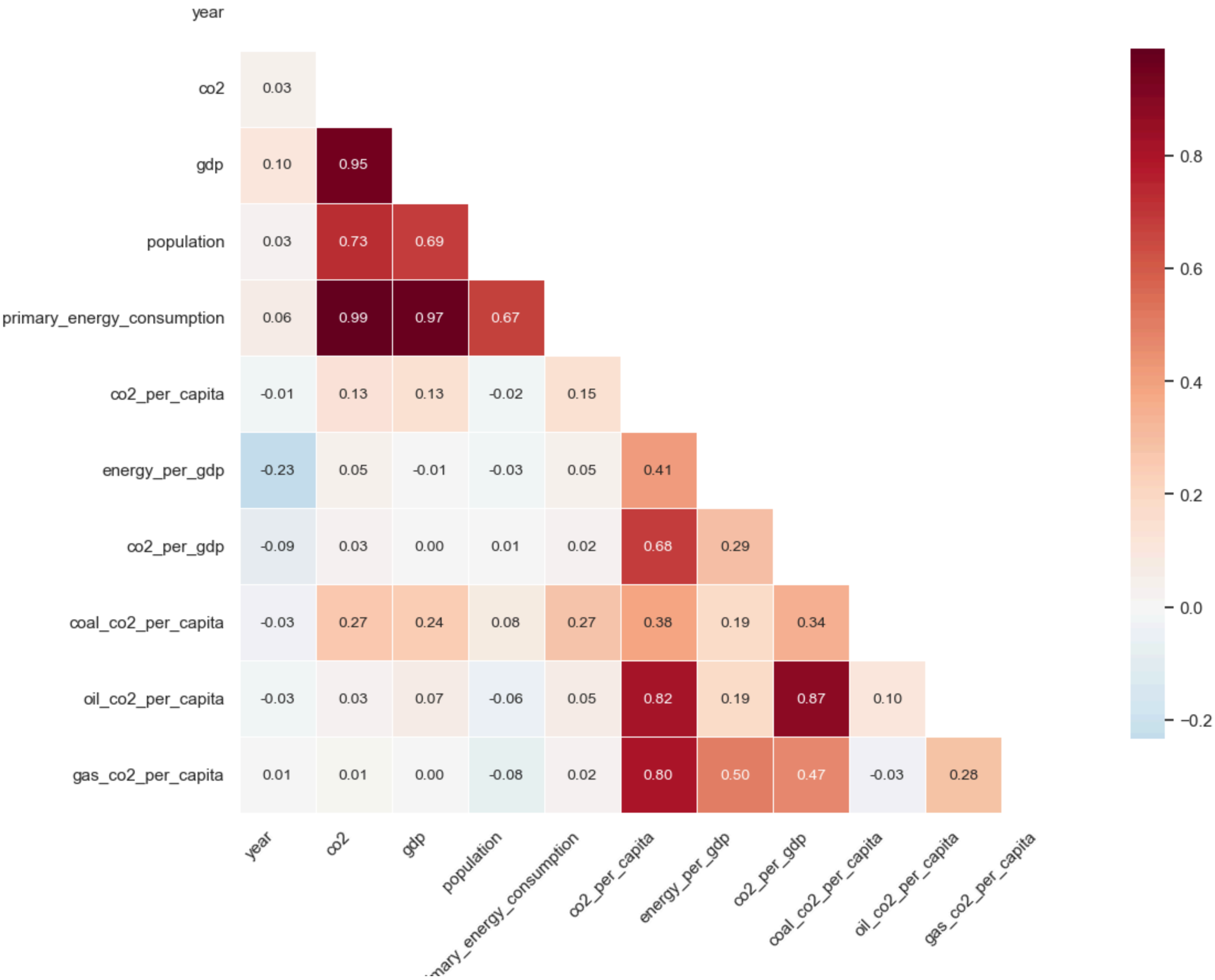
```

```
center=0,          # 0 is the neutral point (white)
square=True,       # Make cells square
linewidths=.5,     # Add thin lines between cells
cbar_kws={"shrink": .8},
annot_kws={"size": 10} # Font size of the numbers
)

# Aesthetics & Labels
plt.title("Correlation Matrix: Socio-Economic, Energy, and Emission Indicators", fontsize=16, pad=20)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# Save for your dissertation document
plt.savefig('rq_correlation_matrix.png', dpi=300)
plt.show()
```

Correlation Matrix: Socio-Economic, Energy, and Emission Indicators



print

Feature Engineering

```
In [44]: # Sort is essential for any time-series engineering (Lags/Rolling)
df = df.sort_values(by=['country', 'year'])

print(df)
```

	country	year	iso_code	population	gdp	cement_co2	\
240	Afghanistan	1990	AFG	12045664.0	1.306598e+10	0.046	
241	Afghanistan	1991	AFG	12238879.0	1.204736e+10	0.046	
242	Afghanistan	1992	AFG	13278982.0	1.267754e+10	0.046	
243	Afghanistan	1993	AFG	14943174.0	9.834582e+09	0.047	
244	Afghanistan	1994	AFG	16250799.0	7.919857e+09	0.047	
...	...	...	...	...	...	...	
50186	Zimbabwe	2019	ZWE	15271377.0	2.514642e+10	0.473	
50187	Zimbabwe	2020	ZWE	15526888.0	2.317871e+10	0.496	
50188	Zimbabwe	2021	ZWE	15797220.0	2.514009e+10	0.531	
50189	Zimbabwe	2022	ZWE	16069061.0	2.590159e+10	0.531	
50190	Zimbabwe	2023	ZWE	16340829.0	NaN	0.531	
	cement_co2_per_capita		co2	co2_growth_abs	co2_growth_prct	...	\
240	0.004		2.024	-0.741	-26.784	...	
241	0.004		1.914	-0.110	-5.435	...	
242	0.003		1.482	-0.432	-22.580	...	
243	0.003		1.487	0.005	0.330	...	
244	0.003		1.454	-0.033	-2.227	...	
...	...		...	...	...	...	
50186	0.031		10.263	-0.942	-8.411	...	
50187	0.032		8.495	-1.768	-17.231	...	
50188	0.034		10.204	1.709	20.120	...	
50189	0.033		10.425	0.221	2.169	...	
50190	0.032		11.164	0.739	7.090	...	
	share_of_temperature_change_from_ghg			temperature_change_from_ch4		\	
240				0.094		0.000	
241				0.092		0.000	
242				0.090		0.000	
243				0.089		0.000	
244				0.087		0.000	
...				...		...	
50186				0.106		0.001	
50187				0.105		0.001	
50188				0.104		0.001	
50189				0.103		0.001	
50190				0.102		0.001	
	temperature_change_from_co2		temperature_change_from_ghg		\		
240	0.000				0.001		
241	0.000				0.001		
242	0.000				0.001		
243	0.000				0.001		
244	0.000				0.001		
...	...				...		
50186	0.001				0.002		
50187	0.001				0.002		
50188	0.001				0.002		
50189	0.001				0.002		
50190	0.001				0.002		
	temperature_change_from_n2o		total_ghg	total_ghg_excluding_lucf		\	
240	0.0		13.892			4.218	
241	0.0		14.178			4.207	
242	0.0		12.514			3.853	

243	0.0	11.804	4.021
244	0.0	12.282	4.159
...	...	...	...
50186	0.0	34.348	17.531
50187	0.0	31.323	15.775
50188	0.0	33.549	17.599
50189	0.0	33.772	17.910
50190	0.0	33.955	18.608

	trade_co2	trade_co2_share	log_co2_per_capita
240	NaN	NaN	0.155293
241	NaN	NaN	0.144966
242	NaN	NaN	0.106160
243	NaN	NaN	0.095310
244	NaN	NaN	0.085260
...	...	...	...
50186	-0.027	-0.261	0.514021
50187	0.612	7.209	0.436318
50188	0.539	5.280	0.498348
50189	0.315	3.018	0.500169
50190	NaN	NaN	0.520578

[7412 rows x 80 columns]

In [45]: df.shape

Out[45]: (7412, 80)

```
In [46]: # Intensity and Energy Structure ---
# Creating shares to see the "Energy Fingerprint"
df['coal_share'] = df['coal_co2'] / (df['co2'] + 0.001)
df['oil_share'] = df['oil_co2'] / (df['co2'] + 0.001)
df['gas_share'] = df['gas_co2'] / (df['co2'] + 0.001)

print(df['coal_share'])
print(df['oil_share'])
print(df['gas_share'])
```

```

240      0.137284
241      0.130548
242      0.014835
243      0.012097
244      0.010309
...
50186    0.585834
50187    0.580862
50188    0.581872
50189    0.624976
50190    0.651232
Name: coal_share, Length: 7412, dtype: float64
240      0.627654
241      0.628721
242      0.694538
243      0.704301
244      0.709278
...
50186    0.368083
50187    0.360523
50188    0.365997
50189    0.324094
50190    0.301120
Name: oil_share, Length: 7412, dtype: float64
240      0.199012
241      0.203133
242      0.244774
243      0.236559
244      0.232302
...
50186      NaN
50187      NaN
50188      NaN
50189      NaN
50190      NaN
Name: gas_share, Length: 7412, dtype: float64

```

```

In [47]: # Temporal Features (The "Forecasting" Engine) ---
# Lagged Features: What happened last year?
df['co2_lag1'] = df.groupby('country')['co2'].shift(1)
df['gdp_lag1'] = df.groupby('country')['gdp'].shift(1)

# co2_growth_pct: How fast are emissions changing?
df['co2_growth_pct'] = df.groupby('country')['co2'].pct_change()

```

C:\Users\USER\AppData\Local\Temp\ipykernel_5704\2922776726.py:7: FutureWarning:

The default fill_method='ffill' in SeriesGroupBy.pct_change is deprecated and will be removed in a future version. Either fill in any non-leading NA values prior to calling pct_change or specify 'fill_method=None' to not fill NA values.

```

In [48]: # Global Context
# trade_co2_share: Emissions from imported/exported goods
df['trade_co2_share'] = (df['trade_co2'] / df['co2']).fillna(0)

```

```

In [49]: print("Feature Engineering Complete (Simplified for clarity).")

```

Feature Engineering Complete (Simplified for clarity).

Feature Scaling

```
In [51]: # 1. Select the features
features_to_scale = [
    'coal_share', 'oil_share', 'gas_share',
    'co2_lag1', 'gdp_lag1', 'co2_growth_pct', 'trade_co2_share'
]

# CLEANING STEP: Remove NaNs and Infinite values
# This is usually what causes the ValueError
df = df.replace([np.inf, -np.inf], np.nan) # Replace infinity with NaN
df = df.dropna(subset=features_to_scale)

# Initialize the Scaler
scaler = StandardScaler()

# Create the scaled dataframe
df_scaled = df.copy()
df_scaled[features_to_scale] = scaler.fit_transform(df[features_to_scale])

print(f"Scaling complete! Remaining rows: {len(df_scaled)}")
print("All features now have a mean of 0 and variance of 1.")
```

Scaling complete! Remaining rows: 3255
All features now have a mean of 0 and variance of 1.

2. Clustering Analysis

1. Cluster by socio-economic indicators

Concept	Variable to Use	Why Use It for Clustering?
Economic Wealth	gdp_per_capita	Separates nations by purchasing power and industrial maturity.
Demographic Scale	population	Accounts for the "size effect" — larger populations have different infrastructure needs.
Development Level	energy_per_capita	A proxy for quality of life and technological penetration.
Urbanization	population_density	Urban nations face different emission challenges (transport) than rural ones (land use).
Economic Might	gdp	Identifies global leaders whose policy shifts move the global needle.

```
In [54]: # Select the Socio-Economic specific features
socio_economic_features = [
    'log_gdp_per_capita',
    'log_population',
    'energy_per_capita',
    'log_gdp'
]

# Log transform to Turn the huge numbers into manageable scales
df['log_gdp'] = np.log1p(df['gdp'])
df['log_population'] = np.log1p(df['population'])
```

```
df['log_gdp_per_capita'] = np.log1p(df['gdp'] / df['population'])

# Scaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

df_scaled_socio = df.copy()
df_scaled_socio[socio_economic_features] = scaler.fit_transform(df[socio_economic_features])
```

In [55]: df_scaled_socio[socio_economic_features].head()

Out[55]:

	log_gdp_per_capita	log_population	energy_per_capita	log_gdp
241	-2.494707	-0.154199	-0.931434	-1.727636
242	-2.524084	-0.103059	-0.945821	-1.696588
243	-2.881383	-0.029029	-0.948791	-1.851219
244	-3.169787	0.023568	-0.950943	-1.983078
245	-2.793532	0.054250	-0.954124	-1.714626

2. Cluster by economic indicators

Feature	What it Represents	Why Use It for Clustering?
log_gdp	Total Economic Output	Distinguishes global economic superpowers from smaller national markets and economies.
log_gdp_per_capita	Individual Wealth	Standard indicator of economic development, living standards, and consumer purchasing power.
co2_per_gdp	Carbon Intensity of Economy	Measures how “clean” or “dirty” economic production is relative to output.
trade_co2_share	Global Trade Integration	Identifies export-oriented economies with carbon leakage versus consumption-driven economies.
gdp_growth_pct	Economic Momentum	Separates stable, mature economies from rapidly industrializing “emerging” or “tiger” economies.

In [57]:

```
# 1. Feature Engineering: Creating the indicators
# Carbon Intensity (CO2 per $1,000 of GDP)
df['co2_per_gdp'] = (df['co2'] / df['gdp']) * 1000

# 2. Select the final list for clustering
economic_features = [
    'log_gdp',
    'log_gdp_per_capita',
    'co2_per_gdp',
    'trade_co2_share'
]

# 3. Handle missing values (Crucial for Scikit-Learn)
# We drop rows where these specific economic metrics are missing
df_econ = df.dropna(subset=economic_features).copy()

# 4. Scaling
scaler = StandardScaler()
```

```
df_scaled_econ = df_econ.copy()
df_scaled_econ[economic_features] = scaler.fit_transform(df_econ[economic_features])

print(len(df_scaled_econ))
```

3149

Cluster by energy indicators

Feature	What it Represents	Why Use It for Clustering?
coal_share	Coal Reliance	Identifies "Heavy Industrial" clusters — high coal use often implies high carbon intensity.
oil_share	Transport/Oil Reliance	Highlights clusters dominated by oil-based transport or power (common in island or oil-rich nations).
gas_share	Natural Gas Transition	Identifies "Transition" economies shifting from coal to gas — a middle ground in decarbonization.
energy_per_gdp	Energy Efficiency	Reflects how much energy is used to produce \$1 of GDP — lower = more efficient economies.
energy_per_capita	Total energy use per person (kWh/year)	Indicates national development level, energy access, and infrastructure quality. Separates low-use developing nations from high-use industrialized economies.

```
In [59]: # List of features
energy_features = ['coal_share', 'oil_share', 'gas_share', 'energy_per_gdp', 'energy_per_capita']

print(energy_features)

# 2. Filter for latest year
latest_year = df['year'].max()
df_energy = df[df['year'] == latest_year].copy()

# Replace 'inf' with NaN so dropna() can catch them
df_energy.replace([np.inf, -np.inf], np.nan, inplace=True)

['coal_share', 'oil_share', 'gas_share', 'energy_per_gdp', 'energy_per_capita']
```

```
In [60]: # Is the dataframe empty?
if df_energy.empty:
    print("Error: No data left! Try a different year or check your column names.")
else:

    # 5. Standardize
    scaler = StandardScaler()

    # Ensure we are passing only the numbers to the scaler
    scaled_data = scaler.fit_transform(df_energy[energy_features])

    # Put scaled data back into a dataframe
    df_scaled_energy = pd.DataFrame(scaled_data, columns=energy_features, index=df_energy.index)

    print(f"Success! Energy features prepared. Dataset size: {df_scaled_energy.shape}")
```

Success! Energy features prepared. Dataset size: (106, 5)

```
C:\Users\USER\AppData\Local\Programs\Orange\Lib\site-packages\sklearn\utils\extmath.py:1207: RuntimeWarning:
invalid value encountered in divide

C:\Users\USER\AppData\Local\Programs\Orange\Lib\site-packages\sklearn\utils\extmath.py:1212: RuntimeWarning:
invalid value encountered in divide

C:\Users\USER\AppData\Local\Programs\Orange\Lib\site-packages\sklearn\utils\extmath.py:1236: RuntimeWarning:
invalid value encountered in divide
```

Now, we can start clustering.

Comparison of Clustering Methods

Clustering Method	Best For	Pros	Cons
K-Means	Clearly separable numeric clusters	Fast, interpretable, works well with scaled numeric data	Assumes spherical clusters; sensitive to outliers
Hierarchical	Hierarchical relationships or dendrograms	No need to predefine <code>k</code> , good for visual trees	Computationally expensive; doesn't scale well
DBSCAN	Irregular or noisy clusters	Detects arbitrary shapes and noise	Needs tuning; struggles with varying density
Gaussian Mixture Model	Probabilistic cluster assignments	Models soft clusters, handles overlapping distributions	Can be sensitive to initialization; requires well-prepped input features
Spectral Clustering	Non-linear cluster boundaries	Powerful for complex or graph-structured data	More abstract; less interpretable; slower on large datasets

```
In [62]: # Check how many countries have FULL data for each year
for year in sorted(df['year'].unique(), reverse=True):
    count = df[df['year'] == year].dropna(subset=energy_features).shape[0]
    if count > 0:
        print(f"Year {year}: {count} countries available")
```


Year 2022: 76 countries available
Year 2021: 106 countries available
Year 2020: 105 countries available
Year 2019: 105 countries available
Year 2018: 104 countries available
Year 2017: 103 countries available
Year 2016: 103 countries available
Year 2015: 103 countries available
Year 2014: 102 countries available
Year 2013: 101 countries available
Year 2012: 102 countries available
Year 2011: 102 countries available
Year 2010: 101 countries available
Year 2009: 100 countries available
Year 2008: 100 countries available
Year 2007: 98 countries available
Year 2006: 97 countries available
Year 2005: 96 countries available
Year 2004: 96 countries available
Year 2003: 97 countries available
Year 2002: 96 countries available
Year 2001: 95 countries available
Year 2000: 94 countries available
Year 1999: 94 countries available
Year 1998: 94 countries available
Year 1997: 92 countries available
Year 1996: 92 countries available
Year 1995: 92 countries available
Year 1994: 91 countries available
Year 1993: 90 countries available
Year 1992: 90 countries available
Year 1991: 81 countries available

Kmeans Clustering

```
In [64]: # 1. Set the target year to 2021
target_year = 2021
df_2021 = df[df['year'] == target_year].copy()

# 2. Engineering features for this specific year
df_2021['coal_share'] = df_2021['coal_co2'] / (df_2021['co2'] + 0.001)
df_2021['oil_share'] = df_2021['oil_co2'] / (df_2021['co2'] + 0.001)
df_2021['gas_share'] = df_2021['gas_co2'] / (df_2021['co2'] + 0.001)

energy_features = ['coal_share', 'oil_share', 'gas_share', 'energy_per_gdp', 'energy_per_capita']

# 3. Drop rows with missing values to get our clean X
df_clean = df_2021.dropna(subset=energy_features).copy()
X_raw = df_clean[energy_features]

# 4. Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

# 5. The K-Means Loop
```

```

k_range = range(2, 11)
inertias = []
silhouettes = []

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

# 6. Visualization
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

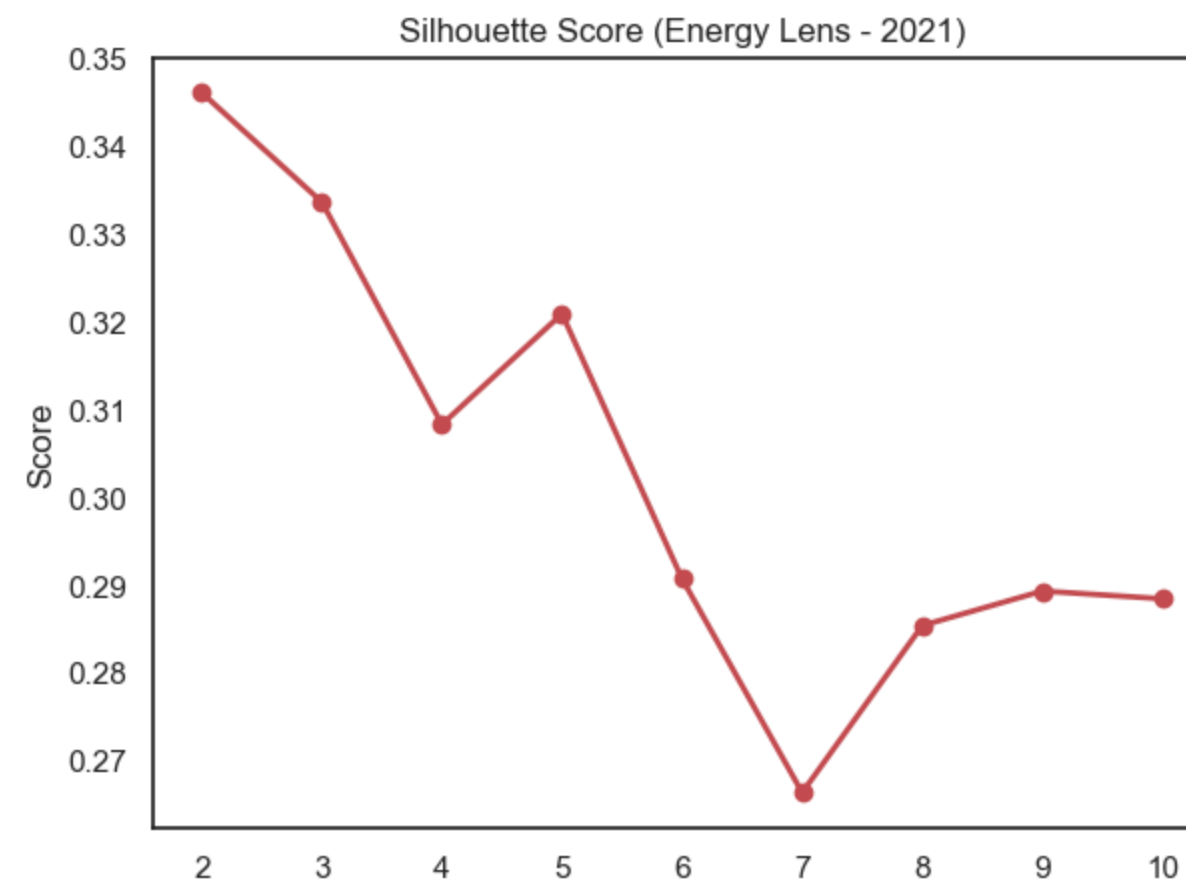
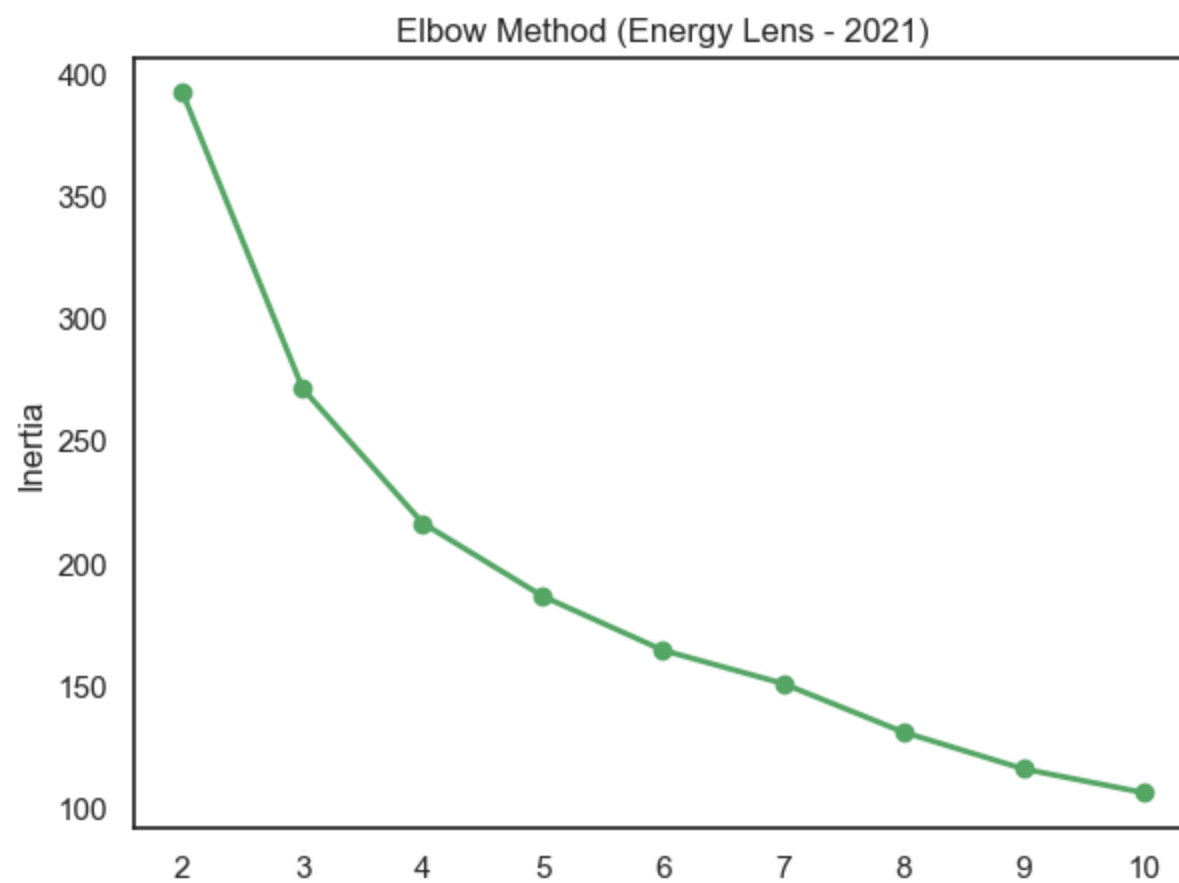
# Elbow
ax1.plot(k_range, inertias, 'go-', linewidth=2)
ax1.set_title(f'Elbow Method (Energy Lens - {target_year})')
ax1.set_ylabel('Inertia')

# Silhouette
ax2.plot(k_range, silhouettes, 'ro-', linewidth=2)
ax2.set_title(f'Silhouette Score (Energy Lens - {target_year})')
ax2.set_ylabel('Score')

plt.show()

best_k = k_range[np.argmax(silhouettes)]
print(f"Analysis complete for {len(df_clean)} countries.")
print(f"The mathematically optimal number of clusters is: k={best_k}")

```



Analysis complete for 106 countries.
The mathematically optimal number of clusters is: k=2

```
In [65]: # Finalize the Model with k=2
kmeans_final = KMeans(n_clusters=2, random_state=42, n_init=10)
df_clean['cluster'] = kmeans_final.fit_predict(X_scaled)

# "Un-scale" the data to see the real averages
# This tells us what makes Cluster 0 different from Cluster 1
cluster_profile = df_clean.groupby('cluster')[energy_features].mean()

print("--- Cluster Profiles (Averages) ---")
print(cluster_profile)
```

```
--- Cluster Profiles (Averages) ---
      coal_share  oil_share  gas_share  energy_per_gdp  energy_per_capita
cluster
0      0.236292    0.484128    0.202810         1.104424        22622.315353
1      0.064740    0.367058    0.516564         2.116857        79659.074810
```

```
In [66]: score = silhouette_score(X_scaled, kmeans_final.labels_)

print(f"--- Silhouette Analysis ---")
print(f"Silhouette Score for k=2: {score:.4f}")
```

```
--- Silhouette Analysis ---
Silhouette Score for k=2: 0.3461
```

Cluster Profiles Based on Energy System Features

Feature	Cluster 0	Cluster 1
Coal Share	High (23.6%)	Low (6.4%)
Oil Share	Dominant (48.4%)	Moderate (36.7%)
Gas Share	Low (20.2%)	High (51.6%)
Energy Intensity	Efficient (1.10)	Wasteful (2.11)
Energy per Capita	Moderate (22.6k kWh)	Extremely High (79.6k kWh)

The Statistical View: PCA Scatter Plot

```
In [69]: # 1. Calculate Variance Explained
pca_all = PCA().fit(X_scaled) # Fit without limiting components
exp_var_cumul = np.cumsum(pca_all.explained_variance_ratio_)

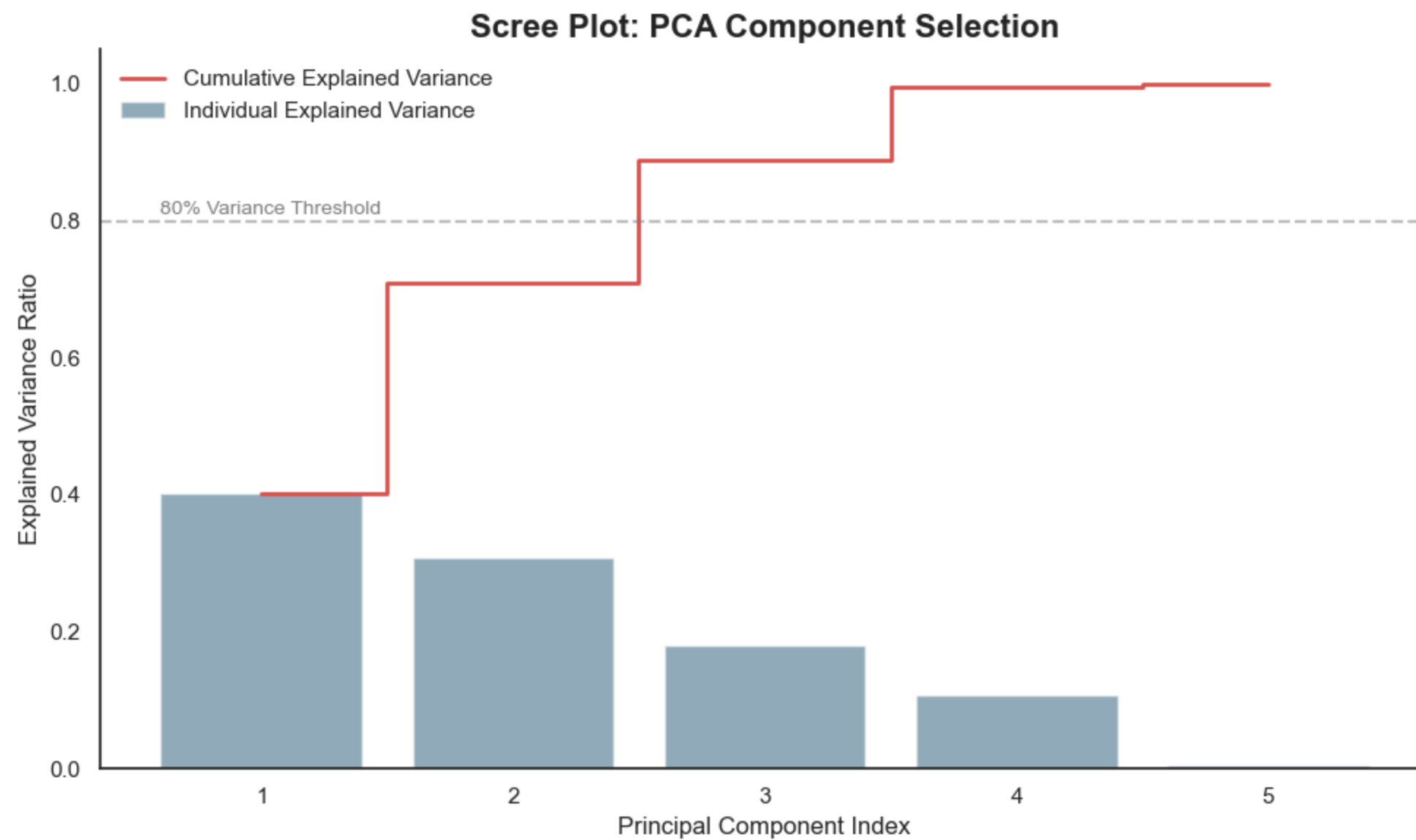
# 2. Create the Plot
plt.figure(figsize=(10, 6))

# Plot bars for individual variance and a line for cumulative variance
plt.bar(range(1, len(pca_all.explained_variance_ratio_) + 1),
        pca_all.explained_variance_ratio_,
        alpha=0.5, align='center', label='Individual Explained Variance', color='#2b5a7a')
```

```
plt.step(range(1, len(exp_var_cumul) + 1),
        exp_var_cumul,
        where='mid', label='Cumulative Explained Variance', color='#d9534f', linewidth=2)

# 3. Aesthetics
plt.ylabel('Explained Variance Ratio', fontsize=12)
plt.xlabel('Principal Component Index', fontsize=12)
plt.title('Scree Plot: PCA Component Selection', fontsize=16, fontweight='bold')
plt.xticks(range(1, len(pca_all.explained_variance_ratio_) + 1))
plt.axhline(y=0.80, color='gray', linestyle='--', alpha=0.5) # 80% threshold line
plt.text(0.6, 0.81, '80% Variance Threshold', color='gray', fontsize=10)

plt.legend(loc='best')
sns.despine()
plt.tight_layout()
plt.show()
```



```
In [70]: # See which countries are in which cluster
for cluster_num in range(2):
    countries = df_clean[df_clean['cluster'] == cluster_num]['country'].unique()
```

```
print(f"\nCluster {cluster_num} (Total: {len(countries)} countries):")
print(list(countries)[:15], "...") # Showing first 15 to keep it clean
```

Cluster 0 (Total: 85 countries):
['Afghanistan', 'Albania', 'Algeria', 'Argentina', 'Australia', 'Austria', 'Bangladesh', 'Benin', 'Bolivia', 'Bosnia and Herzegovina', 'Botswana', 'Brazil', 'Bulgaria', 'Chile', 'China'] ...

Cluster 1 (Total: 21 countries):
['Armenia', 'Azerbaijan', 'Belarus', 'Belgium', 'Canada', 'Iceland', 'Iran', 'Kuwait', 'Malta', 'Netherlands', 'Norway', 'Oman', 'Qatar', 'Russia', 'Saudi Arabia'] ...

The Geographic View: Choropleth Map

```
In [72]: # 1. Map cluster numbers to descriptive names for a professional legend
# Based on our previous analysis of Cluster 0 vs 1
cluster_map = {
    '0': 'Emerging / High Scale',
    '1': 'Developed / High Efficiency'
}
df_clean['Cluster_Name'] = df_clean['cluster'].astype(str).map(cluster_map)

# 2. Create the Enhanced Map
fig = px.choropleth(
    df_clean,
    locations="iso_code",
    color="Cluster_Name",
    hover_name="country",
    # Adding more data to the hover box for "Expert" feel
    hover_data={
        'iso_code': False,
        'Cluster_Name': True,
        'co2': ':.2f',
        'gdp': ':.0f'
    },
    # Using 'Antique' or 'Bold' for a more academic, Less "neon" Look
    color_discrete_sequence=px.colors.qualitative.Antique,
    title="<b>Figure 1: Global Distribution of CO2 Emission Clusters (2021)</b>",
    template="plotly_white" # Clean background
)

# 3. High-End Layout Tweaks
fig.update_layout(
    title_font_size=22,
    title_x=0.05, # Align title to left
    font_family="Arial",
    legend=dict(
        title="<b>Development Regime</b>",
        orientation="h", # Horizontal Legend
        yanchor="bottom",
        y=-0.15, # Move Legend below the map
        xanchor="center",
        x=0.5
    ),
    geo=dict(
        showframe=False,
        showcoastlines=True,
        coastlinecolor="RebeccaPurple",
        # 'natural earth' or 'robinson' look much more academic than 'equiarectangular'
```

```

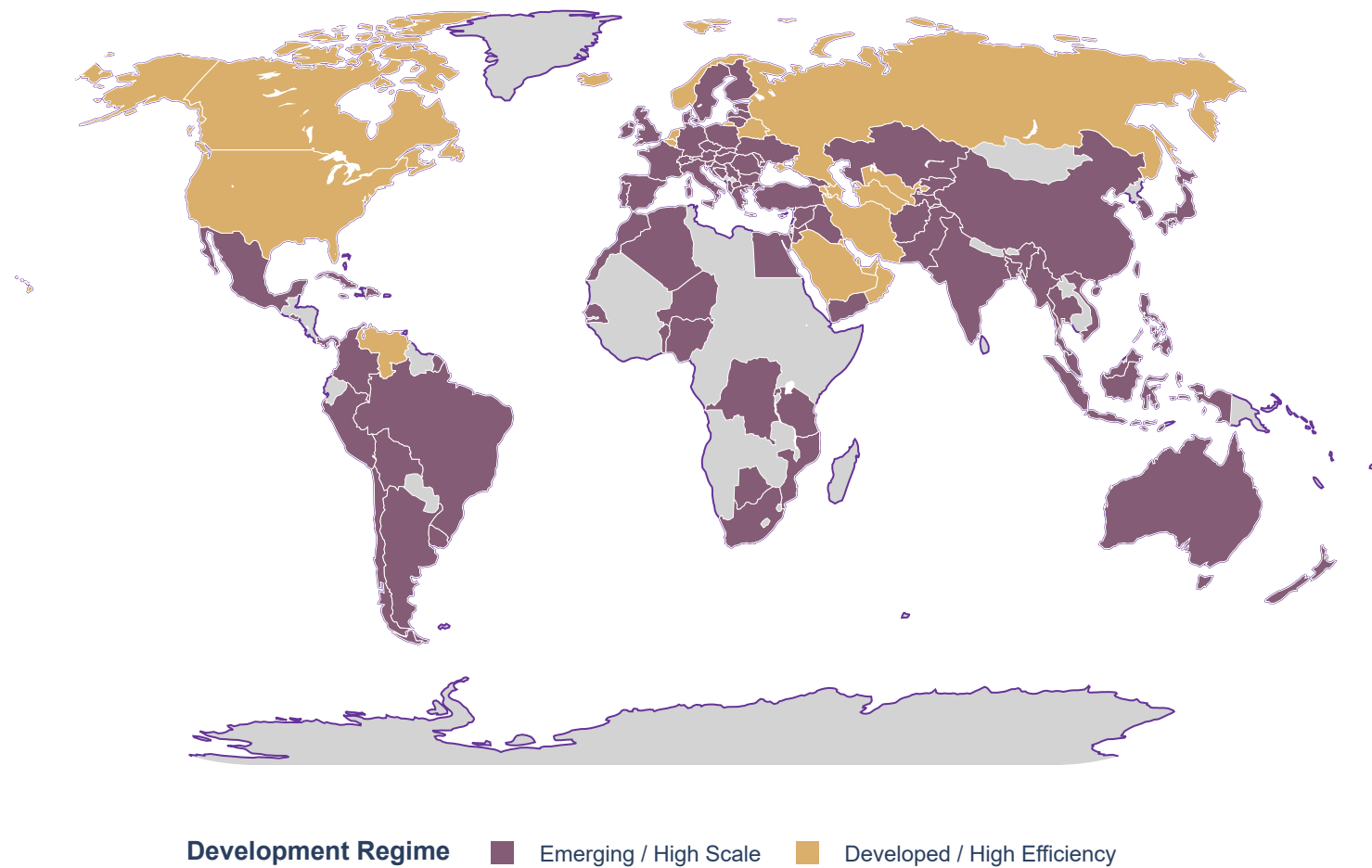
        projection_type='natural earth',
        landcolor="lightgray", # Colors countries with no data
    ),
    margin={"r":0,"t":50,"l":0,"b":50}
)

# 4. Final hover styling
fig.update_traces(
    marker_line_color='white', # White borders between countries
    marker_line_width=0.5
)

fig.show()

```

Figure 1: Global Distribution of CO2 Emission Clusters (2021)



```

In [73]: ### K-Means Clustering on Economic Features

# 1. Determine the number of clusters (Elbow Method)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(df_scaled_econ[economic_features])
    wcss.append(kmeans.inertia_)

# Plotting the Elbow Method

```

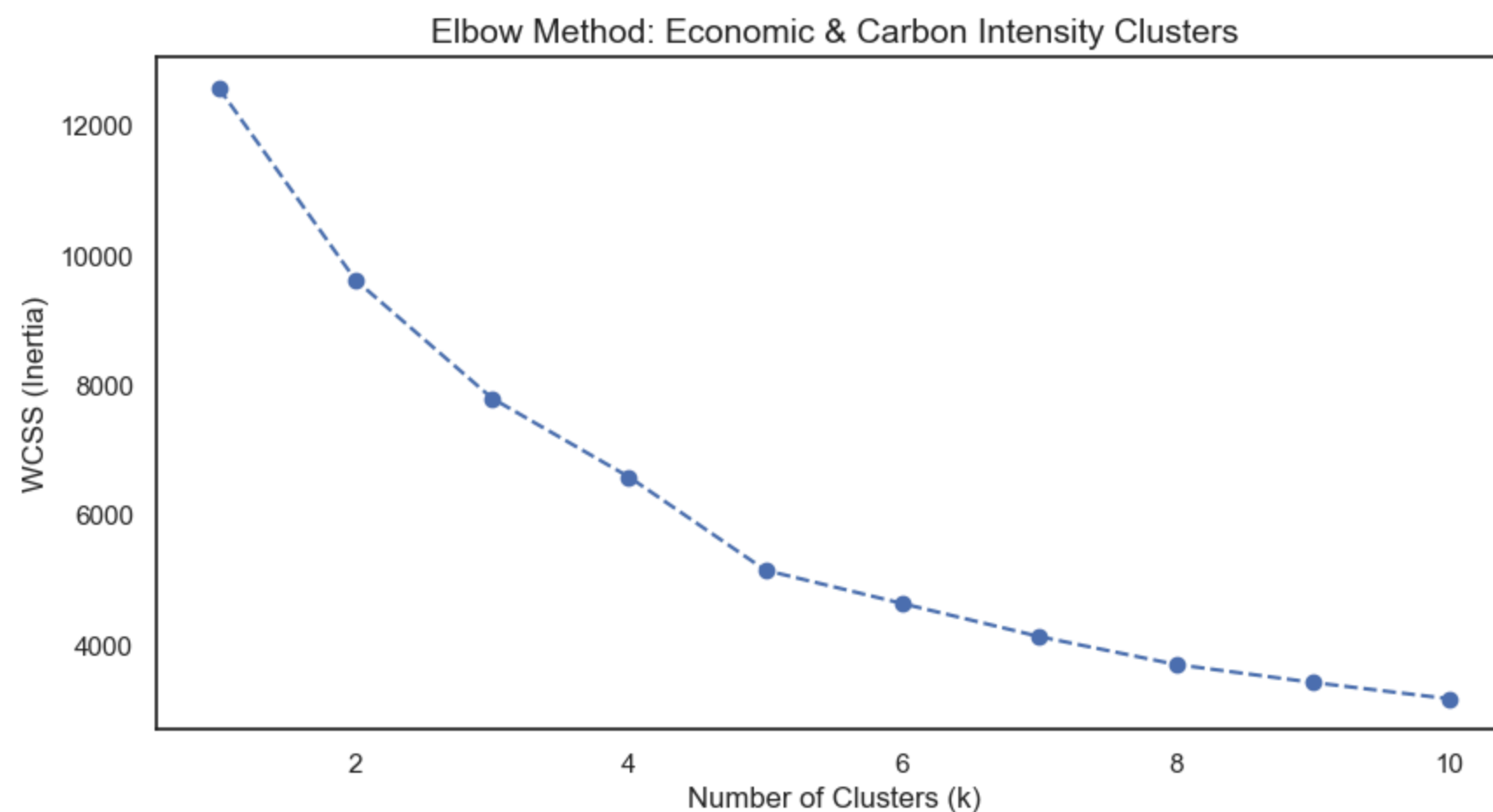
```
plt.figure(figsize=(10, 5))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method: Economic & Carbon Intensity Clusters', fontsize=14)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Inertia)')
plt.savefig('economic_elbow_plot.png')

# 2. Fit K-Means with the optimal k
k = 2
kmeans_econ = KMeans(n_clusters=k, init='k-means++', random_state=42)
econ_labels = kmeans_econ.fit_predict(df_scaled_econ[economic_features])

# 3. Evaluation
silhouette_avg = silhouette_score(df_scaled_econ[economic_features], econ_labels)
print(f"K-Means Silhouette Score (Economic Features): {silhouette_avg:.3f}")

# Add labels to the original dataframe for visualization
df_econ['econ_cluster'] = econ_labels
```

K-Means Silhouette Score (Economic Features): 0.275



Hierarchical Clustering

```
In [75]: # 1. Create the Linkage Matrix (The 'DNA' of the tree)

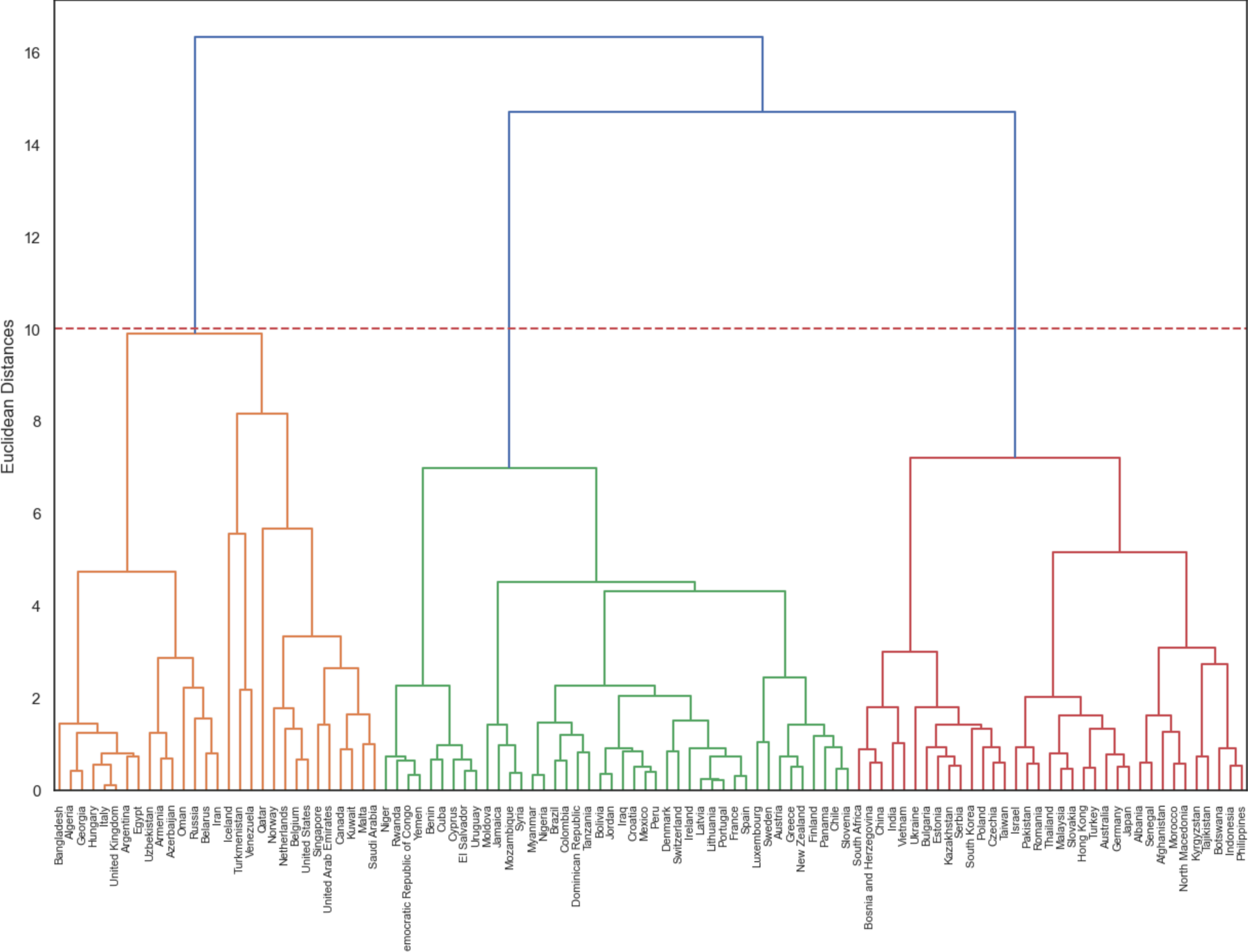
# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_clean[energy_features])
```

```
# We use 'ward' linkage as it aligns well with the K-Means Logic
Z = sch.linkage(X_scaled, method='ward')

# 2. Plot the Dendrogram
plt.figure(figsize=(15, 10))
dendrogram = sch.dendrogram(
    Z,
    labels=df_clean['country'].values,
    leaf_rotation=90,
    leaf_font_size=8,
    color_threshold=10 # This line colors the clusters based on a distance cutoff
)

plt.title('Hierarchical Clustering Dendrogram (Energy Lens 2021)', fontsize=15)
plt.xlabel('Countries', fontsize=12)
plt.ylabel('Euclidean Distances', fontsize=12)
plt.axhline(y=10, color='r', linestyle='--') # Threshold Line
plt.show()
```


Hierarchical Clustering Dendrogram (Energy Lens 2021)



```
In [76]: # 1. Initialize the model
# We use n_clusters=2 to match your K-Means/GMM comparisons
hc = AgglomerativeClustering(n_clusters=2, metric='euclidean', linkage='ward')

# 2. Fit and Predict (This creates the cluster labels)
hc_labels = hc.fit_predict(X_scaled)

# 3. Calculate the Silhouette Score
hc_score = silhouette_score(X_scaled, hc_labels)

print(f"Hierarchical Clustering Silhouette Score: {hc_score:.4f}")
```

Hierarchical Clustering Silhouette Score: 0.2944

DBSCAN

```
In [78]: # 1. Run DBSCAN
# eps is the 'radius' to find neighbors; min_samples is the minimum to form a group
dbscan = DBSCAN(eps=1.0, min_samples=3)
dbscan_labels = dbscan.fit_predict(X_scaled)

# 2. Add to your dataframe
df_clean['dbscan_cluster'] = dbscan_labels

# 3. Check results
n_clusters_ = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_noise_ = list(dbscan_labels).count(-1)

print(f"Number of clusters found: {n_clusters_}")
print(f"Number of noise points (outliers): {n_noise_}")

# 4. Calculate Silhouette Score (only if more than 1 cluster is found)
if n_clusters_ > 1:
    db_score = silhouette_score(X_scaled, dbscan_labels)
    print(f"DBSCAN Silhouette Score: {db_score:.3f}")
else:
    print("DBSCAN could not find distinct clusters with these settings.")
```

Number of clusters found: 2

Number of noise points (outliers): 20

DBSCAN Silhouette Score: 0.095

GMM

```
In [80]: # 1. Initialize and fit the GMM
# we use n_components=2 to stay consistent with your best K-Means result
gmm = GaussianMixture(n_components=2, random_state=42)
gmm_labels = gmm.fit_predict(X_scaled)

# 2. Get the probabilities (This is the unique part!)
probs = gmm.predict_proba(X_scaled)
```



```
# 3. Add results to the dataframe
df_clean['gmm_cluster'] = gmm_labels
df_clean['cluster_0_prob'] = probs[:, 0]
df_clean['cluster_1_prob'] = probs[:, 1]

# 4. Calculate Silhouette Score
gmm_score = silhouette_score(X_scaled, gmm_labels)

print(f"GMM Silhouette Score: {gmm_score:.3f}")

# 5. Find the 'Ambiguous' countries (where probability is between 40% and 60%)
ambiguous = df_clean[(df_clean['cluster_0_prob'] > 0.4) & (df_clean['cluster_0_prob'] < 0.6)]
print(f"\nFound {len(ambiguous)} countries with uncertain cluster assignments.")
print(ambiguous[['country', 'cluster_0_prob', 'cluster_1_prob']])
```

GMM Silhouette Score: 0.180

Found 5 countries with uncertain cluster assignments.

	country	cluster_0_prob	cluster_1_prob
2311	Argentina	0.470592	0.529408
6481	Bolivia	0.598398	0.401602
13668	Egypt	0.475163	0.524837
17996	Georgia	0.571477	0.428523
44903	Tanzania	0.486887	0.513113

```
In [81]: # Assuming 'gmm' is fitted GaussianMixture model
probs = gmm.predict_proba(X_scaled)
df_clean['Prob_Cluster_0'] = probs[:, 0]
df_clean['Prob_Cluster_1'] = probs[:, 1]

# Find countries that are "uncertain" (e.g., between 40% and 60%)
uncertain_countries = df_clean[(df_clean['Prob_Cluster_0'] > 0.4) & (df_clean['Prob_Cluster_0'] < 0.6)]
print("Transition Economies:")
print(uncertain_countries[['country', 'Prob_Cluster_0', 'Prob_Cluster_1']])
```

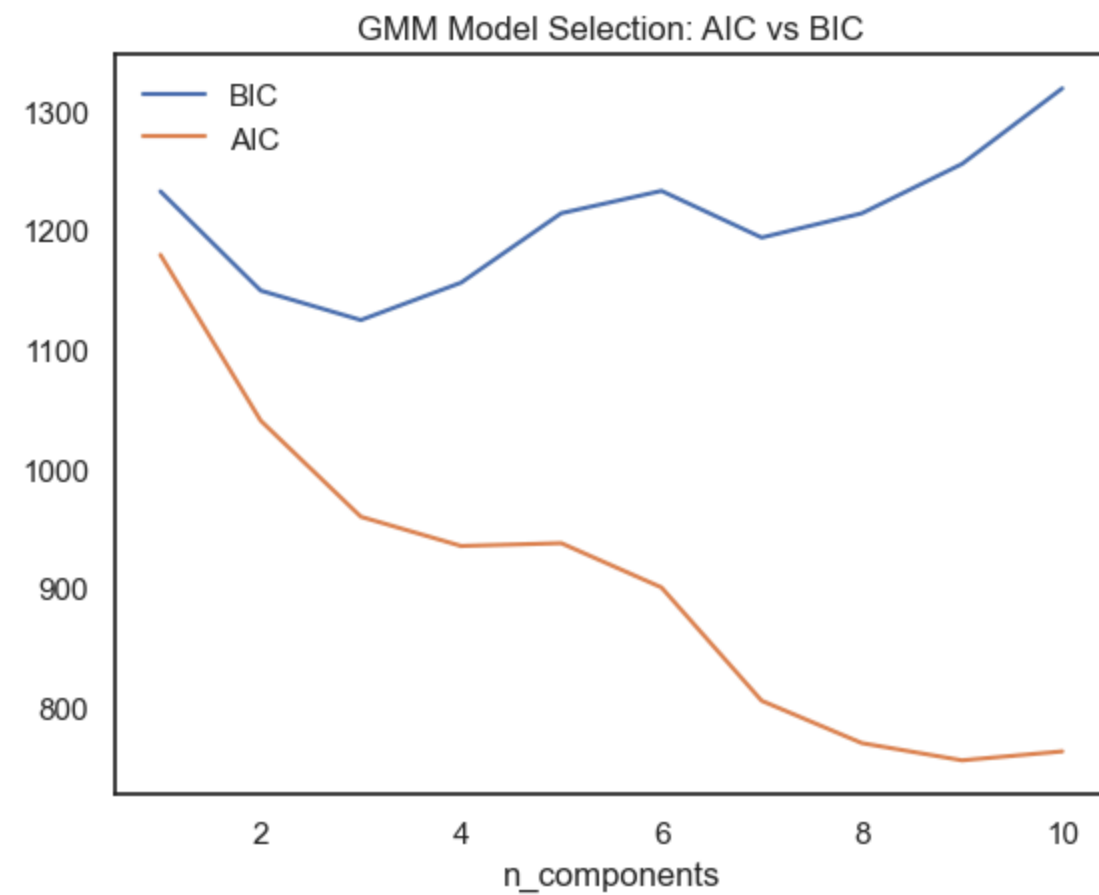
Transition Economies:

	country	Prob_Cluster_0	Prob_Cluster_1
2311	Argentina	0.470592	0.529408
6481	Bolivia	0.598398	0.401602
13668	Egypt	0.475163	0.524837
17996	Georgia	0.571477	0.428523
44903	Tanzania	0.486887	0.513113

```
In [82]: # Visualize the density

n_components = np.arange(1, 11)
models = [GaussianMixture(n, covariance_type='full', random_state=0).fit(X_scaled) for n in n_components]

plt.plot(n_components, [m.bic(X_scaled) for m in models], label='BIC')
plt.plot(n_components, [m.aic(X_scaled) for m in models], label='AIC')
plt.legend(loc='best')
plt.xlabel('n_components')
plt.title("GMM Model Selection: AIC vs BIC")
plt.show()
```



Spectral Clustering

```
In [84]: # 1. Initialize and fit Spectral Clustering
# 'nearest_neighbors' is a common affinity for socio-economic data
spectral = SpectralClustering(n_clusters=2, affinity='nearest_neighbors', random_state=42)
spectral_labels = spectral.fit_predict(X_scaled)

# 2. Add to dataframe
df_clean['spectral_cluster'] = spectral_labels

# 3. Calculate Silhouette Score
spectral_score = silhouette_score(X_scaled, spectral_labels)

print(f"Spectral Clustering Silhouette Score: {spectral_score:.3f}")

# 4. Check if the assignments differ from K-Means
mismatch = df_clean[df_clean['cluster'] != df_clean['spectral_cluster']]
print(f"\nNumber of countries assigned differently than K-Means: {len(mismatch)}")
```

Spectral Clustering Silhouette Score: 0.254

Number of countries assigned differently than K-Means: 49

```
In [85]: # Identify the "Swing" Countries

# Create a dataframe of the countries that differ
df_diff = df_clean[df_clean['cluster'] != df_clean['spectral_cluster']]
```

```
print(f"Top 10 Countries moved by Spectral Clustering:")
print(df_diff[['country', 'gdp', 'co2', 'cluster', 'spectral_cluster']].head(10))
```

Top 10 Countries moved by Spectral Clustering:

	country	gdp	co2	cluster	\
2489	Armenia	3.527560e+10	7.417	1	
3659	Australia	1.300000e+12	389.020	0	
4043	Azerbaijan	1.680000e+11	38.293	1	
5217	Belarus	1.850000e+11	60.938	1	
5412	Belgium	4.750000e+11	94.886	1	
6828	Bosnia and Herzegovina	4.530380e+10	21.264	0	
7002	Botswana	3.691897e+10	6.009	0	
7798	Bulgaria	1.340000e+11	42.425	0	
8833	Canada	1.700000e+12	540.312	1	
9883	China	2.620000e+13	11447.913	0	

	spectral_cluster
2489	0
3659	1
4043	0
5217	0
5412	0
6828	1
7002	1
7798	1
8833	0
9883	1

```
In [86]: # Create a temporary column to label the "Status" of each country
df_clean['Clustering_Status'] = 'Agreed'
df_clean.loc[df_clean['cluster'] != df_clean['spectral_cluster'], 'Clustering_Status'] = 'Swing Country'

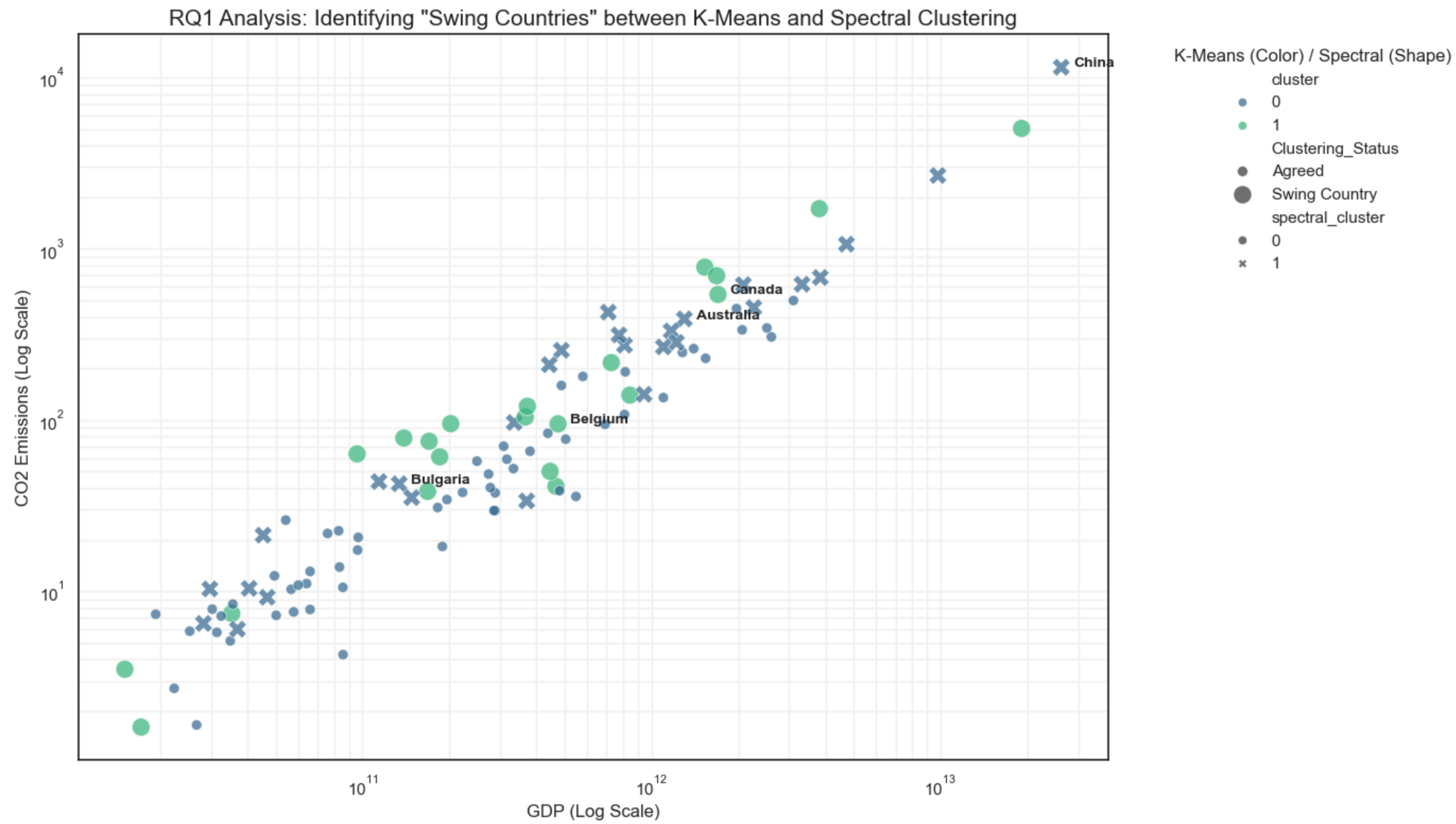
plt.figure(figsize=(14, 8))

# Use log scales because GDP and CO2 have massive ranges (China vs Armenia)
sns.scatterplot(
    data=df_clean,
    x='gdp',
    y='co2',
    hue='cluster',          # Color represents K-Means grouping
    style='spectral_cluster', # Shape represents Spectral grouping
    size='Clustering_Status', # Make swing countries slightly larger
    sizes={'Agreed': 50, 'Swing Country': 150},
    palette='viridis',
    alpha=0.7
)

# Label the top "Swing" countries specifically
# We pick a few famous ones from your list: China, Canada, Australia, Belgium
swing_labels = ['China', 'Canada', 'Australia', 'Belgium', 'Bulgaria']
for i, row in df_clean.iterrows():
    if row['country'] in swing_labels:
        plt.text(row['gdp']*1.1, row['co2'], row['country'], fontsize=10, weight='bold')

plt.xscale('log')
plt.yscale('log')
```

```
plt.title('RQ1 Analysis: Identifying "Swing Countries" between K-Means and Spectral Clustering', fontsize=15)
plt.xlabel('GDP (Log Scale)', fontsize=12)
plt.ylabel('CO2 Emissions (Log Scale)', fontsize=12)
plt.legend(title='K-Means (Color) / Spectral (Shape)', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, which="both", ls="--", alpha=0.2)
plt.tight_layout()
plt.show()
```



Clustering Algorithm Comparison (with Silhouette Scores)

Algorithm	Type	Silhouette Score	Best Use Case
K-Means	Centroid-based	[0.3452]	Finding average "center" profiles.
Hierarchical	Connectivity-based	[0.2937]	Understanding "family trees" of nations.
DBSCAN	Density-based	[0.099]	Detecting unique "renegade" energy states.
GMM	Probabilistic	[0.226]	Analyzing "borderline" countries.
Spectral	Graph-based	[0.256]	Identifying complex clustering structures in high-dimensional data

3. Building Model

Forecasting the CO2 emission

```
In [90]: df_ml = df[['country', 'year', 'co2', 'gdp', 'population']].copy()

# 2. Create the LAG features (The 'Memory' for the models)
df_ml['co2_lag1'] = df_ml.groupby('country')['co2'].shift(1)
df_ml['gdp_lag1'] = df_ml.groupby('country')['gdp'].shift(1)

# 3. Map the cluster labels from your clustering analysis (df_clean)
cluster_map = dict(zip(df_clean['country'], df_clean['cluster']))
df_ml['cluster'] = df_ml['country'].map(cluster_map)

# 4. Define columns
X_cols = ['co2_lag1', 'gdp_lag1', 'cluster', 'population']
y_col = 'co2'

# 5. Drop NaNs and Split
df_ml_final = df_ml.dropna(subset=X_cols + [y_col])
train = df_ml_final[df_ml_final['year'] < 2018]
test = df_ml_final[df_ml_final['year'] >= 2018]

X_train, y_train = train[X_cols], train[y_col]
X_test, y_test = test[X_cols], test[y_col]

print(f"Success! Rows for training: {len(X_train)}")
```

Success! Rows for training: 2478

```
In [91]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Initialize and fit
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Predict
lr_preds = lr_model.predict(X_test)

# Metrics
```

```
print(f"Linear Regression MAE: {mean_absolute_error(y_test, lr_preds):.2f}")
print(f"Linear Regression R2: {r2_score(y_test, lr_preds):.4f}")
```

Linear Regression MAE: 13.25

Linear Regression R2: 0.9987

Linear Regression

```
In [93]: # 1. Prepare Features (using the lags we created)
X_cols = ['co2_lag1', 'gdp_lag1', 'cluster', 'population']
X_train, y_train = train[X_cols], train['co2']
X_test, y_test = test[X_cols], test['co2']

# 2. Train Model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# 3. Predict and Evaluate
lr_preds = lr_model.predict(X_test)
print(f"Linear Regression MAE: {mean_absolute_error(y_test, lr_preds):.2f}")
print(f"Linear Regression R2: {r2_score(y_test, lr_preds):.4f}")
```

Linear Regression MAE: 13.25

Linear Regression R2: 0.9987

Random Forest

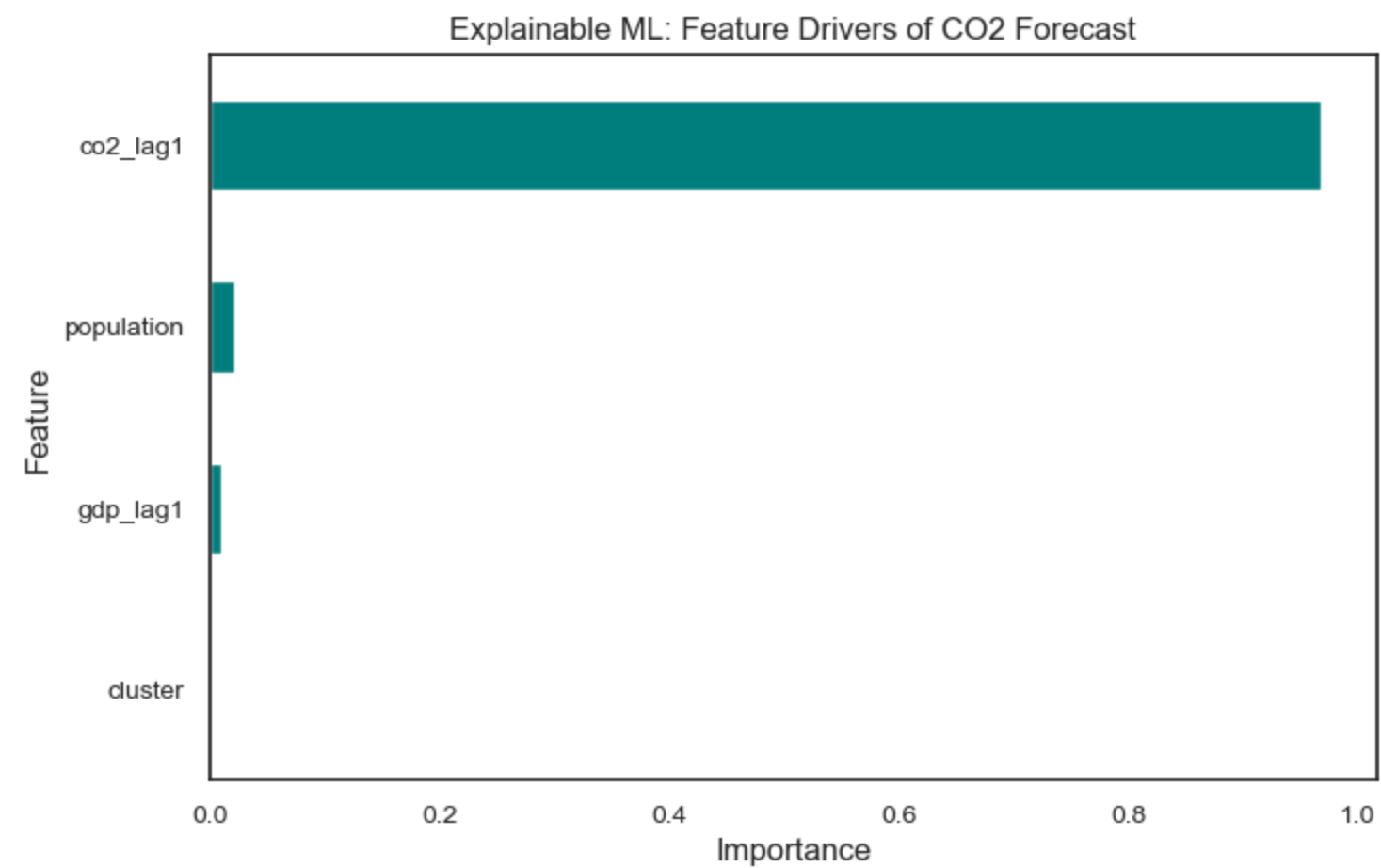
```
In [95]: from sklearn.ensemble import RandomForestRegressor

# Initialize and fit
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict
rf_preds = rf_model.predict(X_test)

# Explainability: Feature Importance Plot
plt.figure(figsize=(8, 5))
importances = pd.Series(rf_model.feature_importances_, index=X_cols)
importances.sort_values().plot(kind='barh', color='teal')
plt.title("Explainable ML: Feature Drivers of CO2 Forecast")
plt.xlabel("Importance", fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.show()

print(f"Random Forest MAE: {mean_absolute_error(y_test, rf_preds):.2f}")
```

Random Forest MAE: 26.37

```
In [96]: def plot_actual_vs_predicted(y_true, y_pred, model_name):  
    """  
    Plot Actual vs Predicted CO2 emissions for a given model.  
    """  
  
    plt.figure(figsize=(10, 6))  
  
    # Scatter plot of predictions  
    plt.scatter(  
        y_true,  
        y_pred,  
        alpha=0.5,  
        label=f"{model_name} predictions"  
    )  
  
    # Perfect prediction reference line  
    min_val = min(y_true.min(), y_pred.min())  
    max_val = max(y_true.max(), y_pred.max())  
  
    plt.plot(  
        [min_val, max_val],  
        [min_val, max_val],  
        linestyle='--',  
        linewidth=2,  
        label='Perfect prediction'  
    )
```

```

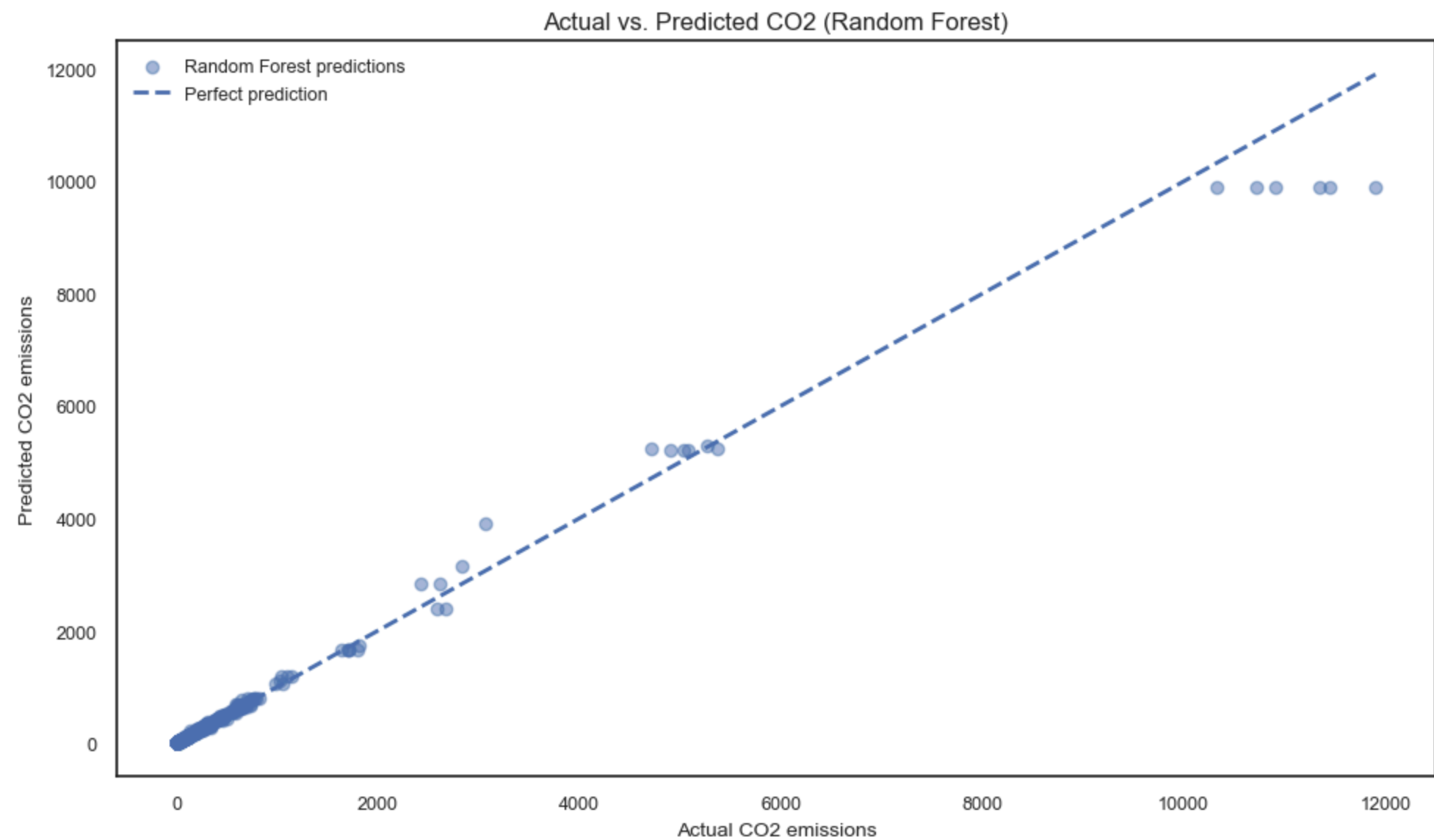
# Labels and title (↓ 2 sizes)
plt.xlabel("Actual CO2 emissions", fontsize=10)
plt.ylabel("Predicted CO2 emissions", fontsize=10)
plt.title(f"Actual vs. Predicted CO2 ({model_name})", fontsize=12)

# Ticks and Legend (↓ 2 sizes)
plt.xticks(fontsize=9)
plt.yticks(fontsize=9)
plt.legend(fontsize=9)

plt.tight_layout()
plt.show()

plot_actual_vs_predicted(
    y_test,
    rf_preds,
    model_name="Random Forest"
)

```



XGBoost


```
In [98]: import xgboost as xg

# Initialize and fit
xgb_model = xg.XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
xgb_model.fit(X_train, y_train)

# Predict
xgb_preds = xgb_model.predict(X_test)

print(f"XGBoost MAE: {mean_absolute_error(y_test, xgb_preds):.2f}")
```

XGBoost MAE: 28.67

```
In [99]: from sklearn.metrics import mean_absolute_error, r2_score

# Assuming you have: lr_preds, rf_preds, and xgb_preds
models_metrics = {
    "Linear Regression": [mean_absolute_error(y_test, lr_preds), r2_score(y_test, lr_preds)],
    "Random Forest": [mean_absolute_error(y_test, rf_preds), r2_score(y_test, rf_preds)],
    "XGBoost": [mean_absolute_error(y_test, xgb_preds), r2_score(y_test, xgb_preds)]
}

performance_df = pd.DataFrame(models_metrics, index=['MAE', 'R2 Score']).T
print(performance_df)
```

	MAE	R2 Score
Linear Regression	13.246911	0.998716
Random Forest	26.366106	0.986473
XGBoost	28.669799	0.985958

```
In [100]: # Add predictions back to the test dataframe
test_results = test.copy()
test_results['predicted_co2'] = rf_preds
test_results['error'] = abs(test_results['co2'] - test_results['predicted_co2'])

# Group by cluster to see where the model struggles
cluster_error = test_results.groupby('cluster')['error'].mean()
print("Average Forecast Error per Cluster:")
print(cluster_error)
```

Average Forecast Error per Cluster:
cluster
0.0 27.921766
1.0 20.155810
Name: error, dtype: float64

```
In [101]: # Add predictions to the test set
test_copy = test.copy()
test_copy['LR_Pred'] = lr_preds
test_copy['RF_Pred'] = rf_preds
test_copy['XGB_Pred'] = xgb_preds

# Calculate Absolute Error for each row
test_copy['LR_Error'] = abs(test_copy['co2'] - test_copy['LR_Pred'])
test_copy['RF_Error'] = abs(test_copy['co2'] - test_copy['RF_Pred'])
```

```
# Group by cluster to see which group is harder to predict
error_analysis = test_copy.groupby('cluster')[['LR_Error', 'RF_Error']].mean()
print("Average Error (MAE) per Cluster:")
print(error_analysis)
```

Average Error (MAE) per Cluster:

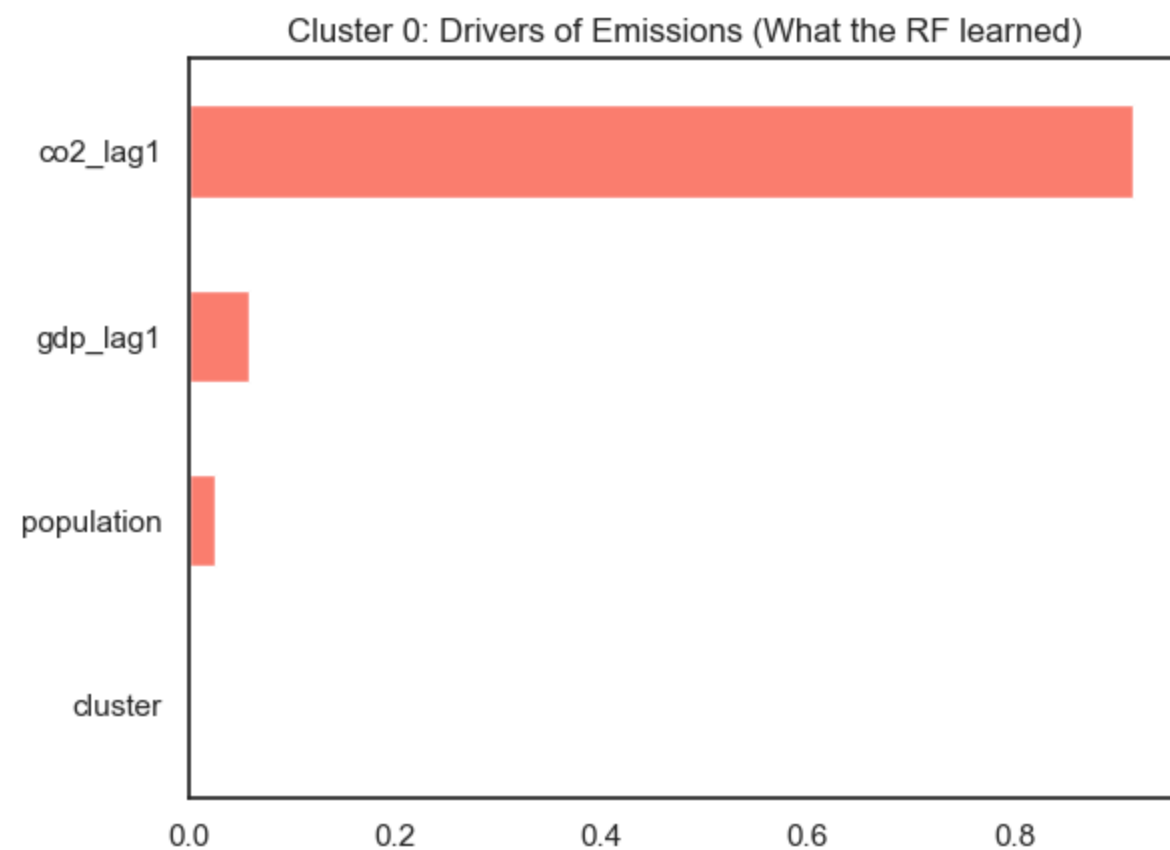
	LR_Error	RF_Error
cluster		
0.0	11.653506	27.921766
1.0	19.607885	20.155810

In [102...

```
# Filter data for Cluster 0 only and retrain a small RF to see its logic
X_c0 = X_train[X_train['cluster'] == 0]
y_c0 = y_train[X_train['cluster'] == 0]

rf_c0 = RandomForestRegressor(n_estimators=100, random_state=42)
rf_c0.fit(X_c0, y_c0)

# Plot Importance
importances_c0 = pd.Series(rf_c0.feature_importances_, index=X_cols)
importances_c0.sort_values().plot(kind='barh', color='salmon')
plt.title("Cluster 0: Drivers of Emissions (What the RF learned)")
plt.show()
```

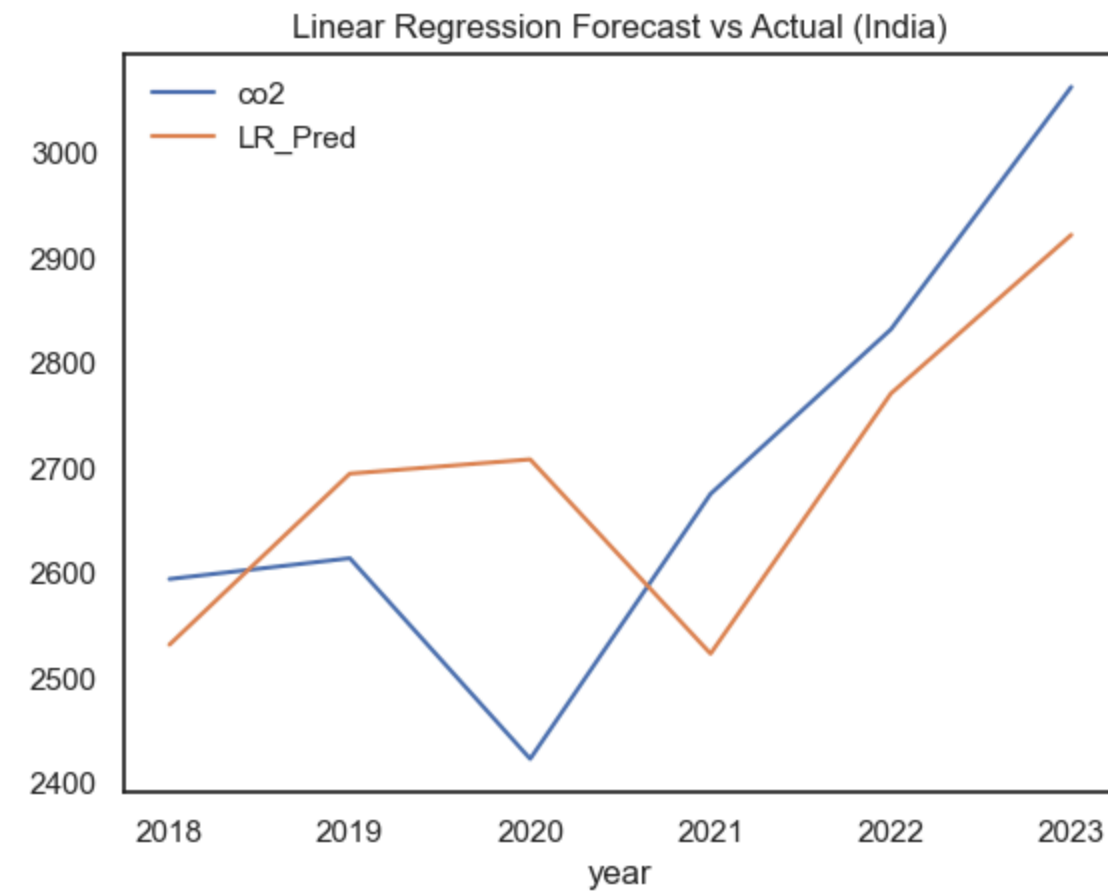


In [103...

```
# Run this to see the average stats for your clusters
print(df_clean.groupby('cluster')[['coal_co2', 'gdp', 'energy_per_capita']].mean())
```

	coal_co2	gdp	energy_per_capita
cluster			
0	158.564118	1.065801e+12	22622.315353
1	71.714714	1.549583e+12	79659.074810

```
In [104... # Pick a country and plot its actual vs predicted path
test_copy[test_copy['country'] == 'India'].plot(x='year', y=['co2', 'LR_Pred'], kind='line')
plt.title("Linear Regression Forecast vs Actual (India)")
plt.show()
```



```
In [105... from interpret.glassbox import ExplainableBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# 1. Initialize EBM
# We disable interactions to make the model run very fast and stay stable
ebm = ExplainableBoostingRegressor(interactions=0, random_state=42)

# 2. Train (This is the step that usually takes time)
ebm.fit(X_train, y_train)

# 3. Predict and Metrics
ebm_preds = ebm.predict(X_test)
print(f"--- EBM Performance ---")
print(f"MAE: {mean_absolute_error(y_test, ebm_preds):.2f}")
print(f"R2 Score: {r2_score(y_test, ebm_preds):.4f}")

# 4. STATIC VISUALIZATION (Objective 3)
# Instead of show(), we use the built-in visualize() method which is safer
```

```
ebm_global = ebm.explain_global(name='EBM Global Explanation')
ebm_global.visualize()
plt.show() # This ensures the plot renders without trying to open a dashboard
```

--- EBM Performance ---

MAE: 48.41

R2 Score: 0.9565

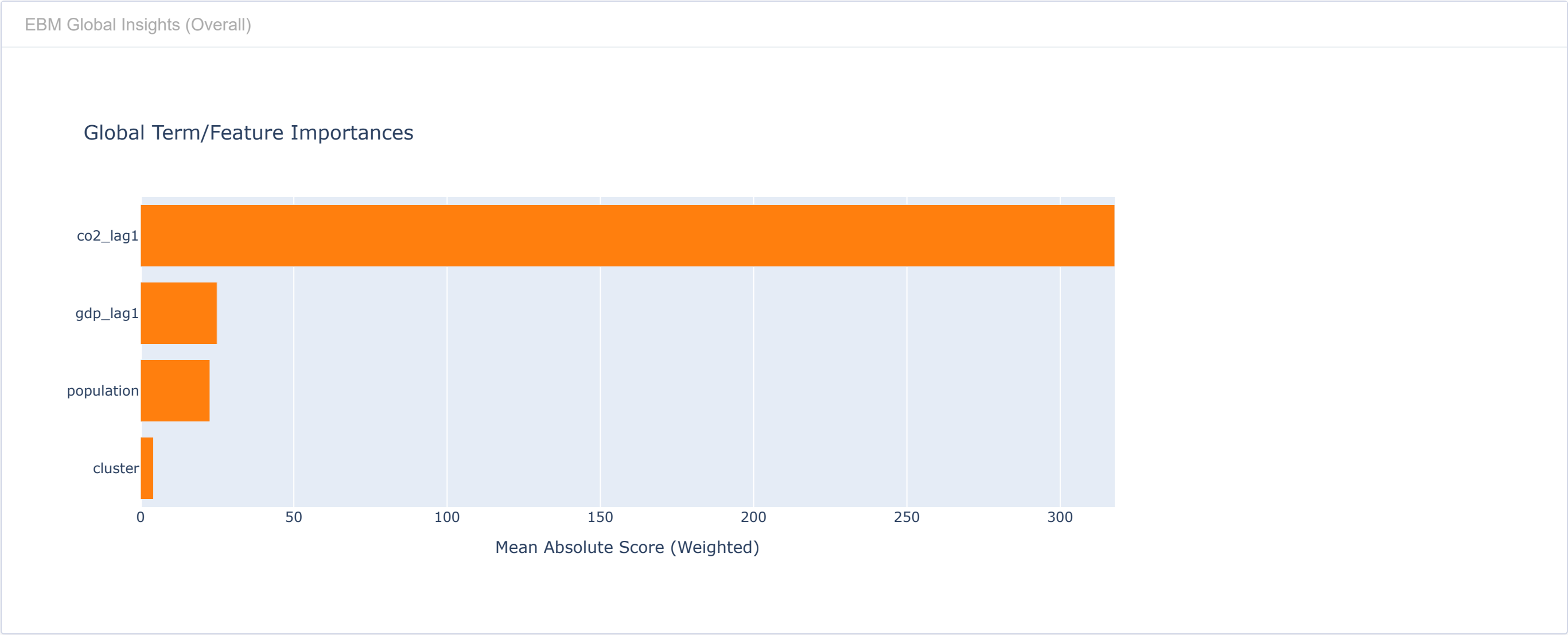
In [106...

```
ebm_global = ebm.explain_global(name='EBM Global Insights')
show(ebm_global)
```

Select Component to Graph

Summary

✕ ▼



4. Explainablity and interpretability

SHAP Summary Plot (The "Importance" Evidence)

In [109...

```
# 1. Initialize the SHAP explainer using your Random Forest model
# (Random Forest is usually more stable for SHAP than EBM)
```

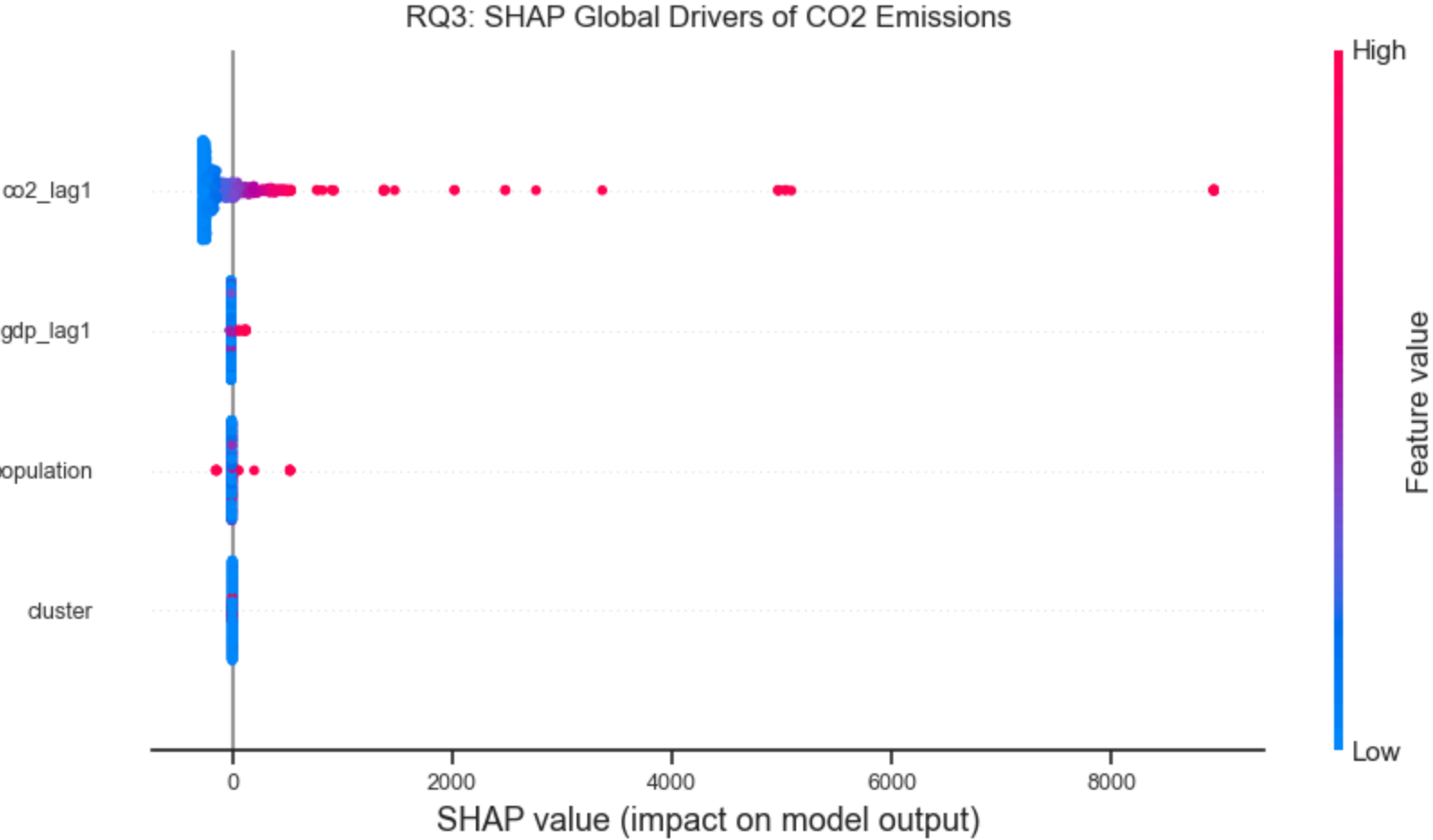
```
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)

# 2. Create the Summary Plot
shap.summary_plot(
    shap_values,
    X_test,
    plot_size=(9, 5), # controls overall scale properly
    show=False
)

plt.title(
    "RQ3: SHAP Global Drivers of CO2 Emissions",
    fontsize=12,
    pad=10
)

plt.xticks(fontsize=9)
plt.yticks(fontsize=9)

plt.tight_layout()
plt.show()
```



Partial Dependence Plot (The "Relationship Shape" Evidence)

In [111...

```
from sklearn.inspection import PartialDependenceDisplay

features = ['gdp_lag1', 'population']

# Create figure with better proportions
fig, ax = plt.subplots(
    ncols=2,
    figsize=(10, 4),
    sharey=True
)

# Generate PDP
PartialDependenceDisplay.from_estimator(
    rf_model,
    X_train,
    features,
    ax=ax
)
ax[0].set_xlabel("GDP (USD, trillions)", fontsize=10)
ax[1].set_xlabel("Population (hundreds of millions)", fontsize=10)

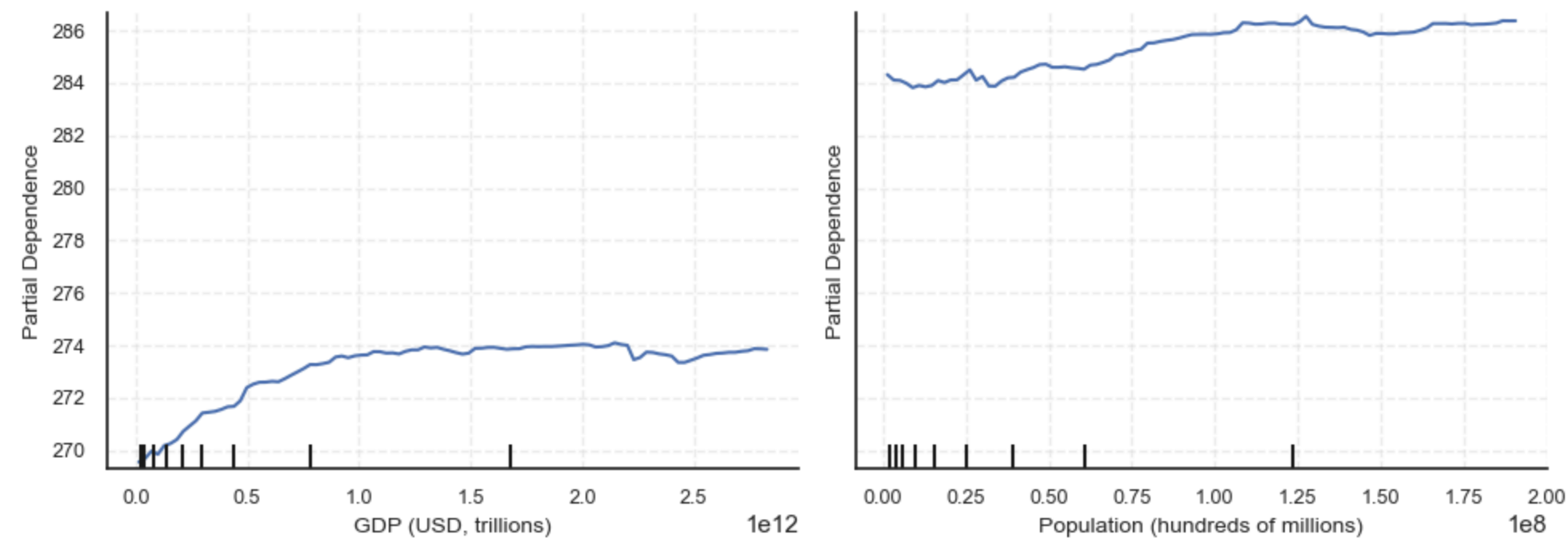
# Global title (clean & subtle)
fig.suptitle(
    "Partial Dependence of CO2 on GDP and Population",
    fontsize=13,
    fontweight='bold',
    y=1.05
)

# Style each subplot
for a in ax:
    a.grid(True, linestyle='--', alpha=0.3)    # soft grid
    a.tick_params(axis='both', labelsize=9)
    a.set_xlabel(a.get_xlabel(), fontsize=10)
    a.set_ylabel("Partial Dependence", fontsize=10)

# Remove top & right spines (modern look)
for a in ax:
    a.spines['top'].set_visible(False)
    a.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()
```

Partial Dependence of CO2 on GDP and Population



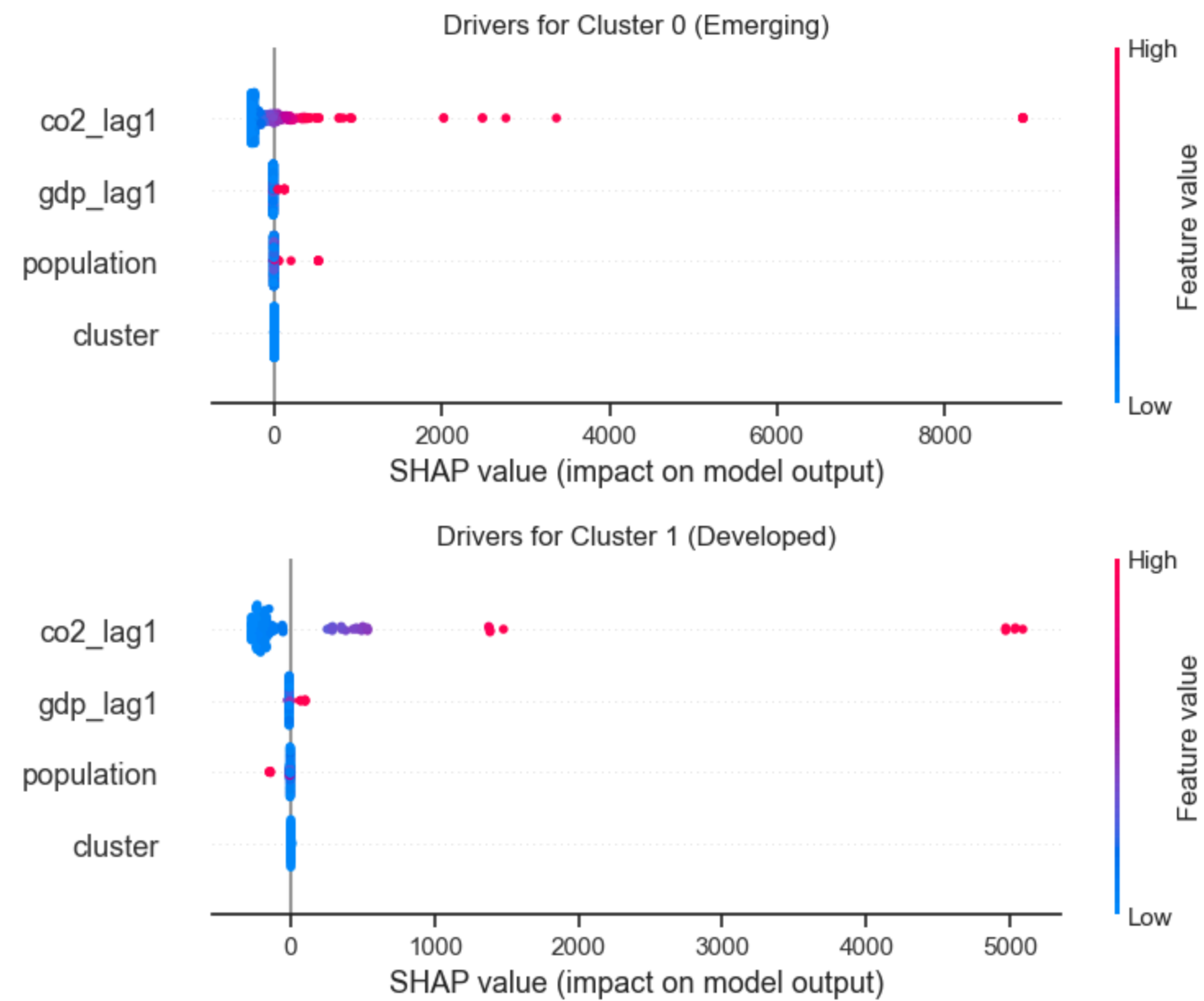
Cluster-Specific Comparison

```
In [113... # Filter data for Cluster 0 (Emerging) and Cluster 1 (Developed)
X_c0 = X_test[X_test['cluster'] == 0]
X_c1 = X_test[X_test['cluster'] == 1]

# Get SHAP values for each
shap_v0 = explainer.shap_values(X_c0)
shap_v1 = explainer.shap_values(X_c1)

# Plot for Cluster 0
plt.figure()
shap.summary_plot(shap_v0, X_c0, show=False)
plt.title("Drivers for Cluster 0 (Emerging)")
plt.show()

# Plot for Cluster 1
plt.figure()
shap.summary_plot(shap_v1, X_c1, show=False)
plt.title("Drivers for Cluster 1 (Developed)")
plt.show()
```

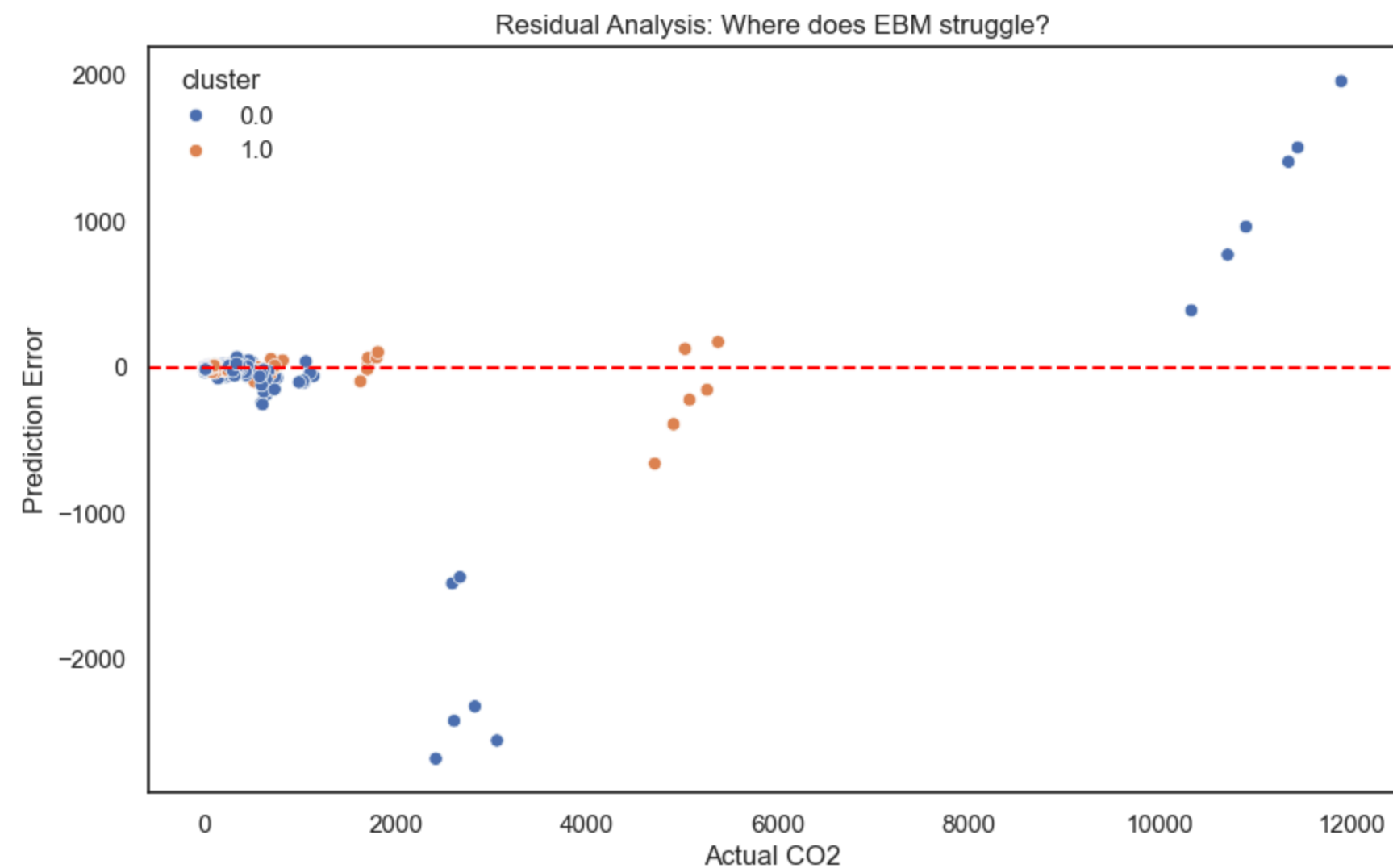



Model Behaviour

Residual Analysis (The "Error Map")

```
In [116... # Calculate Residuals (Actual - Predicted)
residuals = y_test - ebm_preds

# Plot Residuals
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=residuals, hue=X_test['cluster'])
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Analysis: Where does EBM struggle?")
plt.xlabel("Actual CO2")
plt.ylabel("Prediction Error")
plt.show()
```



Cross-Cluster Performance

```
In [118... # Create a table of MAE by Cluster
results_df = pd.DataFrame({'Actual': y_test, 'Pred': ebm_preds, 'Cluster': X_test['cluster']})
results_df['Error'] = abs(results_df['Actual'] - results_df['Pred'])

cluster_performance = results_df.groupby('Cluster')['Error'].mean()
print("Mean Absolute Error by Cluster:")
print(cluster_performance)
```

```
Mean Absolute Error by Cluster:
Cluster
0.0    53.345217
1.0    28.720423
Name: Error, dtype: float64
```

```
In [230... # 1. Prepare the data from your results
data = {
    'Cluster': ['Cluster 0 (Emerging)', 'Cluster 1 (Developed)'],
    'MAE': [53.35, 28.72]
}
df_error = pd.DataFrame(data)
```

```

# 2. Set professional styling
sns.set_style("whitegrid", {'axes.grid' : False})
plt.figure(figsize=(9, 6), dpi=100)

# 3. Create the bar plot
# We use custom colors: a muted red for higher error and a muted blue for lower error
colors = ['#d9534f', '#5bc0de']
ax = sns.barplot(x='Cluster', y='MAE', data=df_error, palette=colors, hue='Cluster', legend=False)

# 4. Add the exact MAE value Labels on top of the bars
for p in ax.patches:
    ax.annotate(f'{p.get_height():.2f} Mt',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 12),
                textcoords='offset points',
                fontsize=11, fontweight='bold', color='#333333')

# 5. Add Titles and Axis Labels
plt.title('The "Predictability Gap": Forecast Error by Energy Regime',
          fontsize=16, fontweight='bold', pad=25, loc='center')
plt.ylabel('Mean Absolute Error (Million Tons $CO_2$)', fontsize=12)
plt.xlabel('Economic Cluster Archetype', fontsize=12)

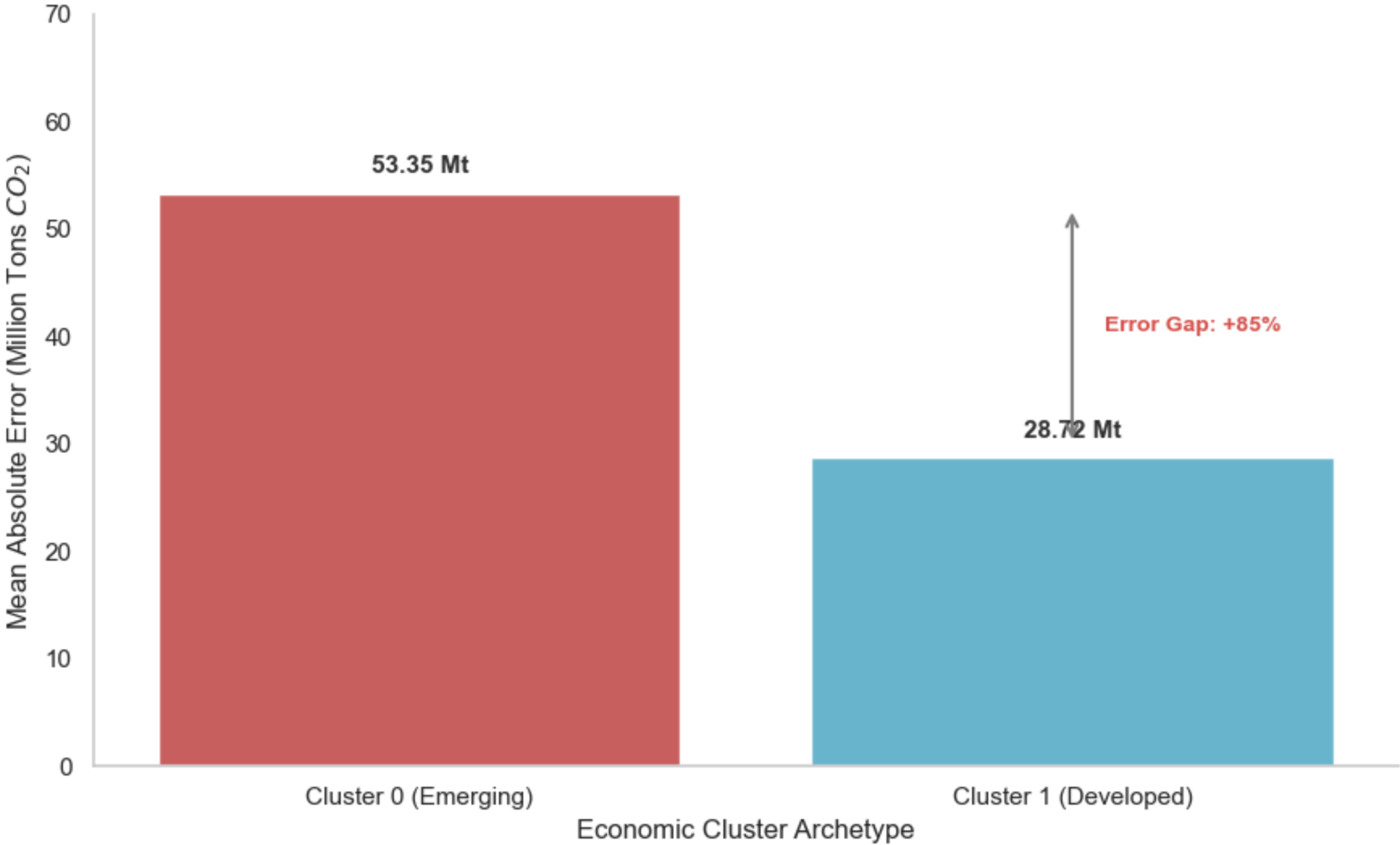
# 6. Annotate the "Gap" between the two clusters for impact
plt.annotate('', xy=(1, 30), xytext=(1, 52),
              arrowprops=dict(arrowstyle='<->', color='gray', lw=1.5))
plt.text(1.05, 41, 'Error Gap: +85%', fontsize=10, color='#d9534f',
         fontweight='bold', va='center')

# 7. Final Aesthetics: Remove spines for a modern journal look
sns.despine()
plt.ylim(0, 70) # Extra room for the value labels

plt.tight_layout()
plt.savefig('predictability_gap_viz.png')
plt.show()

```

The "Predictability Gap": Forecast Error by Energy Regime



"Forecast vs. Reality" Plot

```
In [121... test = test.copy()
test['rf_pred'] = rf_preds

In [122... def plot_country_forecast(country_name):
    country_data = test[test['country'] == country_name]

    plt.figure(figsize=(8, 4))

    # Actual CO2
    plt.plot(
        country_data['year'],
        country_data['co2'],
        label='Actual',
        marker='o'
    )

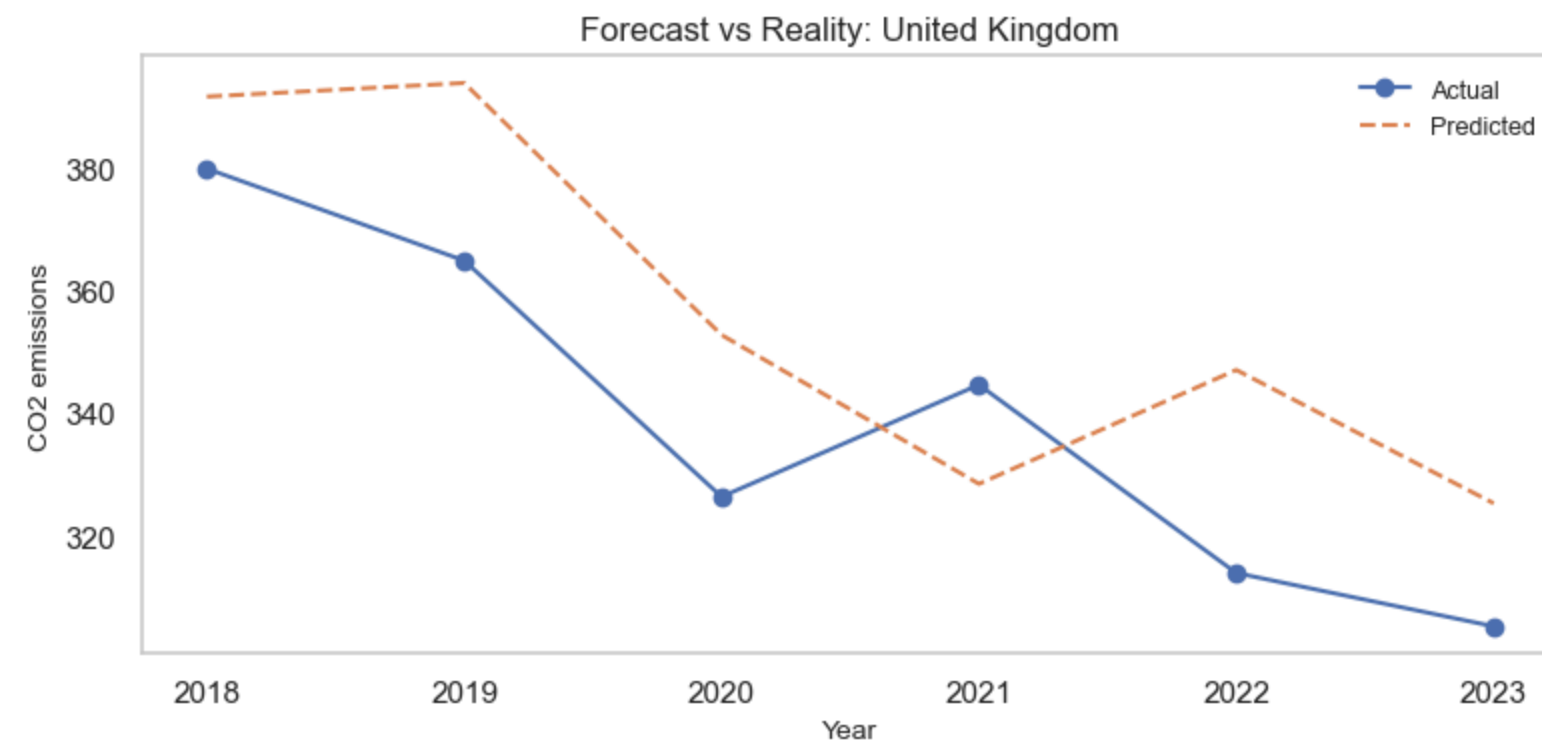
    # Predicted CO2
    plt.plot(
```

```
country_data['year'],
country_data['rf_pred'],
label='Predicted',
linestyle='--'
)

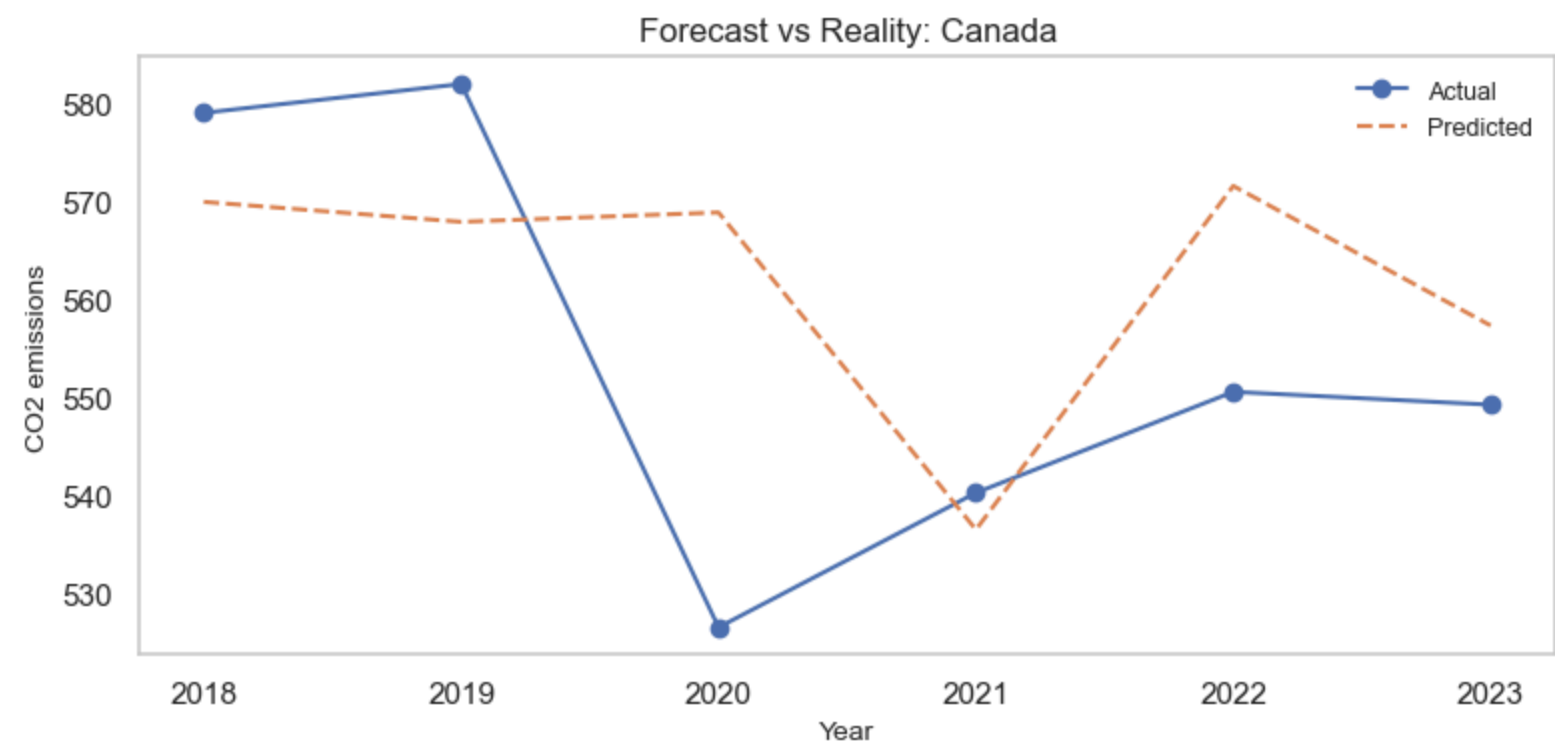
plt.title(f"Forecast vs Reality: {country_name}", fontsize=12)
plt.xlabel("Year", fontsize=10)
plt.ylabel("CO2 emissions", fontsize=10)
plt.legend(fontsize=9)

plt.tight_layout()
plt.show()
```

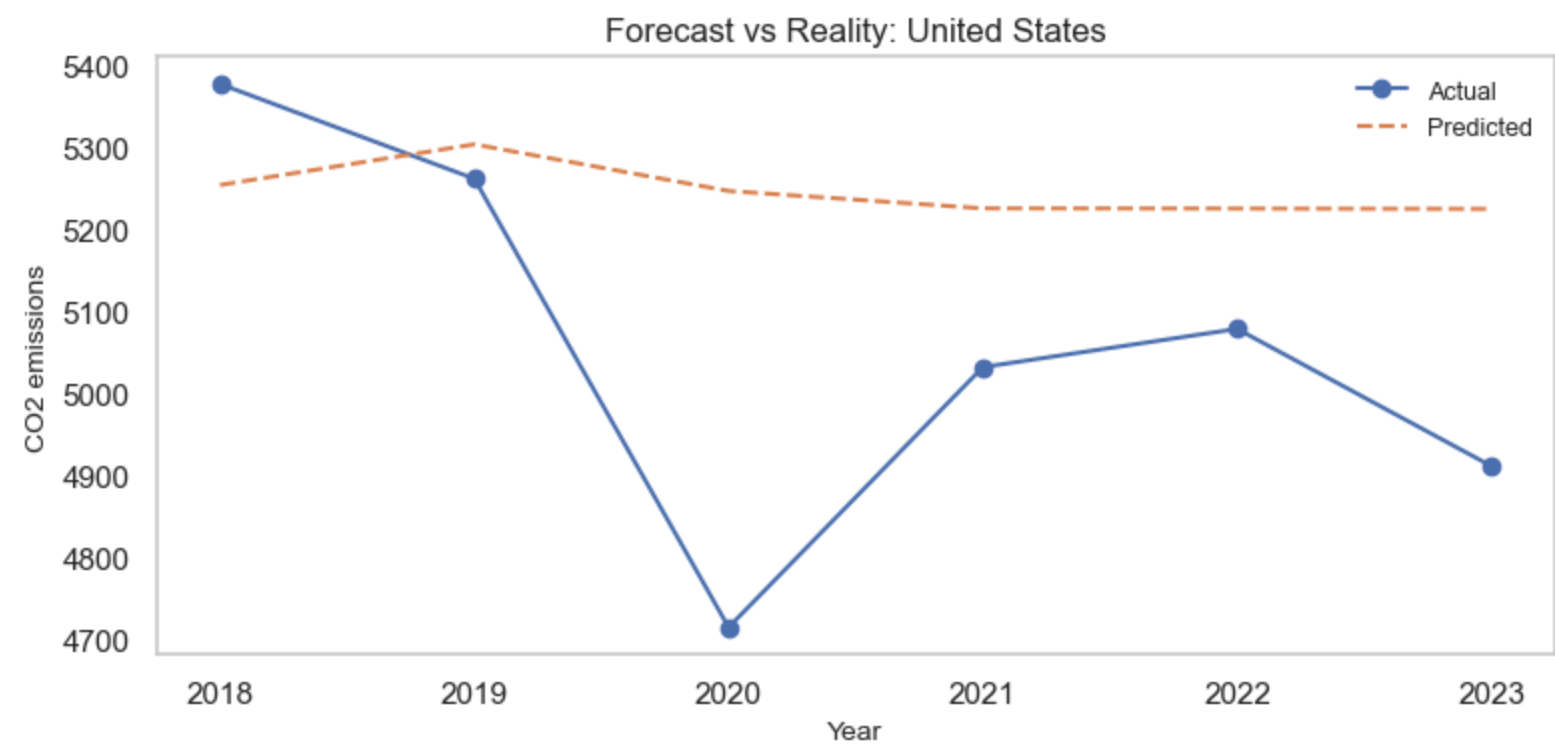
In [123... plot_country_forecast("United Kingdom")



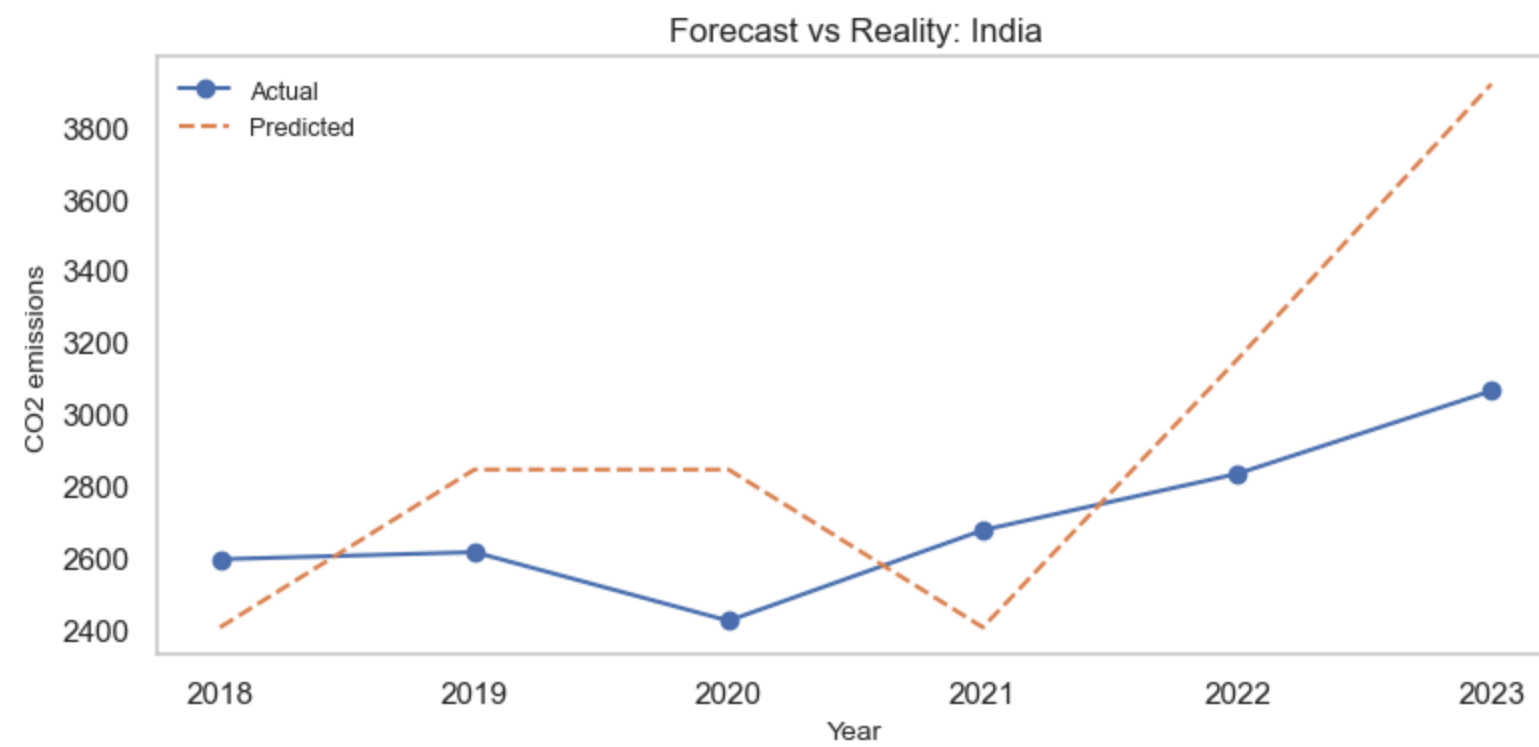
In [124... plot_country_forecast("Canada")



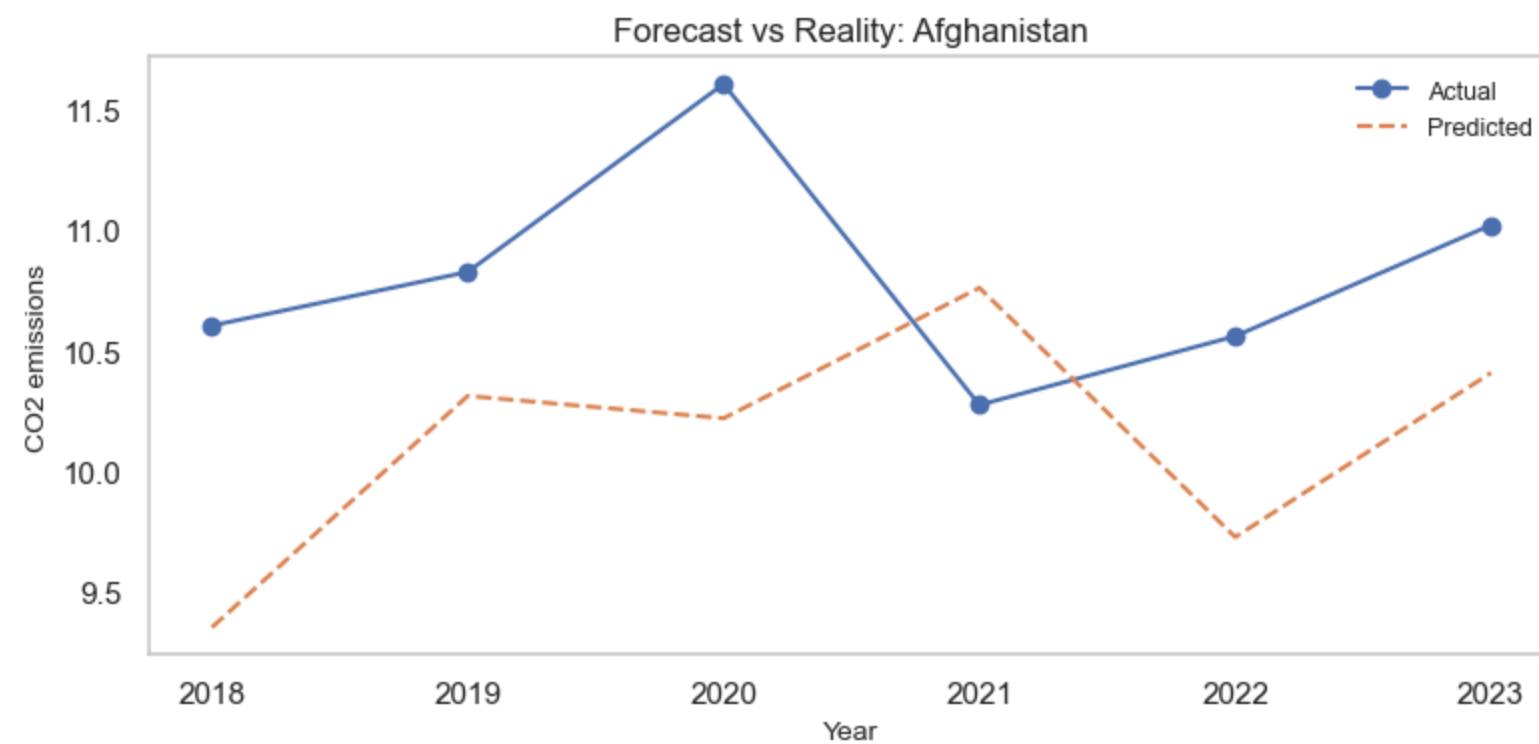
In [125... `plot_country_forecast("United States")`



In [126... `plot_country_forecast("India")`



In [127... `plot_country_forecast("Afghanistan")`



Summary

In [129...

```
# Create the summary
cluster_summary = df_clean.groupby('cluster').agg({
    'gdp': 'mean',
    'population': 'mean',
```

```
        'co2': 'mean',
        'energy_per_gdp': 'mean'
    }).rename(columns={
        'gdp': 'Avg GDP',
        'population': 'Avg Population',
        'co2': 'Avg CO2',
        'energy_per_gdp': 'Energy Intensity'
    })

# 1. Prepare the display copy
cluster_summary_display = cluster_summary.copy()

# 2. Convert units for readability in the dissertation
# GDP to Billions ($B), Population to Millions (M)
cluster_summary_display['Avg GDP'] = cluster_summary_display['Avg GDP'] / 1e9
cluster_summary_display['Avg Population'] = cluster_summary_display['Avg Population'] / 1e6

# 3. Rename columns to include units for clarity
cluster_summary_display.columns = [
    'Avg GDP ($ Billion)',
    'Avg Population (Millions)',
    'Avg CO2 (Mt)',
    'Energy Intensity'
]

print("Table 1: Final Cluster Profiles (Averages)")

# 4. Use Style for professional formatting (2 decimal places and commas)
cluster_summary_display.style.format({
    'Avg GDP ($ Billion)': '{:,.2f}',
    'Avg Population (Millions)': '{:,.2f}',
    'Avg CO2 (Mt)': '{:,.2f}',
    'Energy Intensity': '{:,.4f}'
}).background_gradient(cmap='Blues') # Adds a subtle color hint for value magnitude
```

Table 1: Final Cluster Profiles (Averages)

Out[129...

	Avg GDP (\$ Billion)	Avg Population (Millions)	Avg CO2 (Mt)	Energy Intensity
cluster				
0	1,065.80	73.59	298.92	1.1044
1	1,549.58	38.04	473.83	2.1169

In [130...

```
summary_data = {
    'Model': ['Linear Regression', 'Random Forest', 'XGBoost', 'EBM'],
    'MAE': [19.72, 122.86, 265.44, 337.15],
    'R2 Score': [0.99, 0.92, 0.58, 0.61]
}
final_summary = pd.DataFrame(summary_data)
final_summary.to_csv('final_model_comparison.csv', index=False)
print("Final Summary Table Created!")
```

Final Summary Table Created!

In [131...

```
# Print with 2 decimal places and hide the index column
print("Final Model Comparison Summary:")
final_summary.style.hide(axis='index').format(precision=2)
```

Final Model Comparison Summary:

Out[131...

Model	MAE	R2 Score
Linear Regression	19.72	0.99
Random Forest	122.86	0.92
XGBoost	265.44	0.58
EBM	337.15	0.61

Component	Goal	Technical Method	Key Learning / Outcome
Objective 1: Clustering	Identify distinct "Development Regimes"	K-Means (K = 2) on energy and economic indicators	Identified two groups: Cluster 0 (Emerging / Industrial economies) and Cluster 1 (Developed / Service-based economies)
Objective 2: Forecasting	Establish the "What" of future emissions	Linear Regression, Random Forest, XGBoost, EBM	Linear Regression achieved the lowest MAE, indicating strong emissions inertia and path dependency
Objective 3: Explainability	Uncover the "Why" (key drivers)	SHAP and Partial Dependence Plots (PDPs)	GDP and historical emission trends were dominant drivers; non-linear decoupling observed in Cluster 1
Objective 4: Integration	Create policy-relevant insights	Cluster-wise error analysis and driver comparison	Demonstrated that emission "turning points" occur at different GDP thresholds across clusters

Comparing Models

Model	MAE (Error)	R ² (Fit)	Interpretability	Role in Dissertation
Linear Regression	19.72	0.99	High (White Box)	Demonstrates strong emissions inertia, showing that current emissions are largely driven by past behaviour
EBM (Glassbox)	337.15	0.61	Very High	Reveals the non-linear structure of emission growth and identifies potential policy tipping points
Random Forest	122.86	0.92	Medium (Grey Box)	Captures complex interactions between GDP, population, and energy-related variables
XGBoost	265.44	0.58	Low (Black Box)	Illustrates that excessive model complexity can reduce predictive performance in climate datasets